Escuela Técnica Superior de Ingeniería
Universidad de Huelva

Grado en Ingeniería Informática

Trabajo Fin de Grado

Solución web para el seguimiento y descubrimiento de obras literarias

Autor: Torres Contreras, Juan
Tutor: Álvarez Macías, José Luis
Huelva, Junio 2025

Índice

Resumen 7
Abstract 7
1. Introducción 8
1.1. Motivación 8
1.2. Objetivos 8
1.3. Metodología de desarrollo 9
1.4. Estructura del documento 9
1.5. Estado del arte 10
1.5.1. Plataformas web 10
1.5.2. Extensiones de navegador 10
1.5.3. Posicionamiento del proyecto 10
2. Requisitos del sistema 10
2.1. Scraper 10
2.1.1. Requisitos generales 11
2.1.2. Requisitos funcionales y no funcionales 11
2.1.3. Actores 12
2.1.4 Casos de uso 13
2.2. Página web 18
2.2.1. Requisitos generales 18
2.2.2. Requisitos funcionales y no funcionales 18
2.2.3. Actores 20
2.2.4. Casos de uso 21
2.3. Extensión de navegador 39
2.3.1. Requisitos generales 39
2.3.2. Requisitos funcionales y no funcionales 39
2.3.3. Actores 41
2.3.4. Casos de uso 42
3. Visión general del sistema 47
3.1. Arquitectura y relación entre componentes 47
3.2. Diagrama de componentes 48
4. Tecnologías utilizadas 49
4.1. Herramientas de diseño y modelado 49
4.1.1. Figma 49
4.1.2. Astah UML 49
4.2. Recursos gráficos 49
4.2.1. Flaticon 49

4.2.2. Font Awesome 50
4.3. Tecnologías Web 50
4.3.1. HTML 50
4.3.2. CSS 50
4.3.3. JS 50
4.4. Herramientas de desarrollo 50
4.4.1. Pycharm 50
4.4.2. Github 51
4.5. Frameworks y librerías 51
4.5.1. Django 51
4.5.2. Scrapy 51
4.5.3. Playwright 51
4.6. Bases de datos 52
4.6.1. PostgreSQL 52
4.7. Plataformas de despliegue 52
4.7.1. Render 52
5. Componente 1: Sistema de Web Scraping (Scrapy) 52
5.1. Introducción técnica 52
5.2. Estructura del Scraper 53
5.3. Gestión de errores 54
5.4. Inserción en la base de datos (pipeline) 54
5.5. Proceso para dar soporte a un nuevo sitio 55
6. Componente 2: Plataforma Web (Django) 58
6.1. Introducción técnica 58
6.1.1. Arquitectura del framework (MTV) 59
6.1.2. Sistema de persistencia (ORM) 59
6.1.3. Funcionalidades integradas y ventajas de Django 59
6.2. Interfaces de usuario (prototipos) 60
6.3. Modelado de base de datos 65
6.3.1. Modelo de usuario 66
6.3.2. Relación entre libros y fuentes 66
6.3.3. Capítulos y progreso 66
6.3.4. Otros aspectos del modelo 67
6.4. Interfaz de fusión de libros 68
6.5. Mapas de navegación 69
6.6. Estructura de las plantillas 73
6.7. Limitaciones 74
6.8. API Básica 74

7. Componente 3: Extensión de Navegador (Chromium) 74
7.1. Introducción técnica 74
7.2. Interfaz de usuario 75
7.3. Estructura de la extensión 76
7.4. Proceso para dar soporte a un nuevo sitio 77
7.5. Limitaciones técnicas y soluciones adoptadas 79
8. Integración 79
8.1. Problemas encontrados durante la integración 80
9. Despliegue 80
10. Estimación de costes 81
10.1. Costes de personal 81
10.2. Costes de software 82
10.3. Costes de hardware 82
10.4. Costes de infraestructura y despliegue 82
10.5. Costes operativos 83
10.6. Coste total 83
11. Conclusiones 84
12. Bibliografía 85
13. Anexos 86
13.1. Uso de imágenes con fines educativos 86
13.2. Glosario 86

Índice de tablas

Tabla 1: Scraper - Requisitos generales 11
Tabla 2: Scraper - Requisitos funcionales 11
Tabla 3: Scraper - Requisitos no funcionales 12
Tabla 4: Scraper - Actores 12
Tabla 5: Scraper - CU-01 (Scrapear catálogo completo) 15
Tabla 6: Scraper - CU-02 (Scrapear libro) 17
Tabla 7: Scraper - CU-03 (Validar sitio) 17
Tabla 8: Página web - Requisitos generales 18
Tabla 9: Página web - Requisitos funcionales 19
Tabla 10: Página web - Requisitos no funcionales 20
Tabla 11: Página web - Actores 20
Tabla 12: Página web - CU-01 (Logueo) 23
Tabla 13: Página web - CU-02 (Registro) 24
Tabla 14: Página web - CU-03 (Búsqueda por título) 25
Tabla 15: Página web - CU-04 (Búsqueda avanzada) 27
Tabla 16: Página web - CU-05 (Ver detalles del libro) 27
Tabla 17: Página web - CU-06 (Ver capítulo) 28
Tabla 18: Página web - CU-07 (Agregar/Quitar como favorito) 29
Tabla 19: Página web - CU-08 (Cambiar 'FollowedStatus') 30
Tabla 20: Página web - CU-09 (Cambiar índice de lectura) 31
Tabla 21: Página web - CU-10 (Continuar lectura) 32
Tabla 22: Página web – CU-11 (Ver configuración de la cuenta) 33
Tabla 23: Página web – CU-12 (Seleccionar idomas favoritos) 34
Tabla 24: Página web – CU-13 (Cambiar imagen de perfil) 35
Tabla 25: Página web – CU-14 (Activar/Desactivar notificaciones por correo) 36
Tabla 26: Página web – CU-15 (Ver lista de libros seguidos) 37
Tabla 27: Página web – CU-16 (Cerrar sesión) 37
Tabla 28: Página web - CU-17 (Fusionar libros) 39
Tabla 29: Extensión de navegador - Requisitos generales 39
Tabla 30: Extensión de navegador - Requisitos funcionales 40
Tabla 31: Extensión de navegador - Requisitos no funcionales 41
Tabla 32: Extensión de navegador - Actores 41
Tabla 33: Extensión de navegador - CU-01 (Registrar capítulo leído) 43
Tabla 34: Extensión de navegador - CU-02 (Sincronizar historial de lectura) 45
Tabla 35: Extensión de navegador - CU-03 (Mostrar compatibilidad) 46
Tabla 36: Extensión de navegador - CU-04 (Mostrar información de la extensión) 47
Tabla 37: Flags de comportamiento para la integración con Playwright 57
Tabla 38: Costes de personal 81
Tabla 39: Costes de software 82
Tabla 40: Costes de hardware 82
Tabla 41: Costes de infraestructura y despliegue 83
Tabla 42: Costes operativos 83
Tabla 43: Coste total 84

Índice de figuras

Ilustración 1: Scraper - Diagrama de casos de uso (Administrador) 13
Ilustración 2: Página web - Diagrama de casos de uso (cualquier usuario)..... 21
Ilustración 3: Página web - Diagrama de casos de uso (Usuario No Logueado) 21
Ilustración 4: Página web - Diagrama de casos de uso (Usuario Logueado) 22
Ilustración 5: Página web - Diagrama de casos de uso (Administrador web)..... 22
Ilustración 6: Extensión de navegador - Diagrama de casos de uso..... 42
Ilustración 7: Diagrama de componentes..... 48
Ilustración 8: Ejemplo de elementos a scrapear en la página del catálogo..... 56
Ilustración 9: Ejemplo de elementos a scrapear en la página de detalles..... 56
Ilustración 10: Prototipos de la página principal, del login y del registro de usuario..... 61
Ilustración 11: Prototipos de la carátula de un libro..... 61
Ilustración 12: Prototipos para las vistas de la configuración de la cuenta 62
Ilustración 13: Prototipos para las vistas de los filtros de búsqueda 63
Ilustración 14: Prototipos de las notificaciones y mensajes posibles 64
Ilustración 15: Prototipo de la página de detalles de un libro 64
Ilustración 16: Diagrama UML del modelado de la base de datos..... 65
Ilustración 17: Interfaz del sistema de fusión de libros 68
Ilustración 18: Navegación para el logueo y registro de usuario..... 69
Ilustración 19: Mensaje de advertencia tras hacer una búsqueda sin resultados 70
Ilustración 20: Navegación de un usuario no logueado a la vista de detalles..... 70
Ilustración 21: Navegación de un usuario logueado a MyList y a la página de detalles 71
Ilustración 22: Navegación de un usuario logueado al panel de configuración de la cuenta72
Ilustración 23: Interfaz para cambiar el FollowedStatus 73
Ilustración 24: Popup de la extensión al estar en la página principal..... 75
Ilustración 25: Popup de la extensión al estar en una página web externa 76
Ilustración 26: Icono de la extensión al estar en un sitio web soportado..... 76
Ilustración 27: Icono de la extensión al estar en un sitio web no soportado 76
Ilustración 28: Ejemplo de elementos a extraer de la página de lectura..... 78

Resumen

Actualmente, muchos lectores de cómics en línea utilizan múltiples plataformas para seguir diferentes series, ya que no existe un único servicio que reúna todo el catálogo disponible. Esta fragmentación provoca que el historial de lectura quede disperso, lo que dificulta el seguimiento global del progreso.

Este trabajo presenta una plataforma web que centraliza el historial de lectura del usuario, sincronizando el avance en cada serie de forma automática, independientemente del sitio en el que se haya leído. La solución se compone de tres elementos principales: un sistema de extracción de contenido, una extensión de navegador que detecta automáticamente el capítulo leído y una interfaz web desde la que se accede a toda la información recopilada.

El objetivo final es ofrecer una herramienta que simplifique el seguimiento de lectura y unifique el acceso al historial personal del usuario.

Palabras clave: plataforma web, arquitectura modular, integración de sistemas, sincronización de datos, extensión de navegador, seguimiento automático, historial de lectura, web scraping, Scrapy, Playwright, Django, Python, JavaScript, HTML, CSS, PostgreSQL, backend, frontend.

Abstract

Currently, many online comic readers use multiple platforms to follow different series, as there is no single service that brings together the entire available catalogue. This fragmentation causes reading histories to be scattered, making it difficult to track overall progress.

This project presents a web platform that centralises the user's reading history, automatically synchronising progress in each series, regardless of the site on which it was read. The solution is composed of three main elements: a content extraction system, a browser extension that automatically detects the chapter being read, and a web interface that provides access to all the collected information.

The ultimate goal is to offer a tool that simplifies reading tracking and unifies access to the user’s personal reading history.

Keywords: web platform, modular architecture, system integration, data synchronisation, browser extension, automatic tracking, reading history, web scraping, Scrapy, Playwright, Django, Python, JavaScript, HTML, CSS, PostgreSQL, backend, frontend.

1. Introducción

En los últimos años, la lectura digital de mangas, manhwas y manhuas ha crecido notablemente, apoyada por la proliferación de lectores online, extensiones de navegador (Google, s.f.) y plataformas de seguimiento. Sin embargo, muchas de estas soluciones presentan limitaciones: requieren una actualización manual y/o dependen de catálogos cerrados.

Este TFG se centra en el desarrollo de una solución que combina una plataforma web con una extensión de navegador, diseñada para facilitar el seguimiento automático de la lectura en sitios de terceros. El sistema permite almacenar el progreso del usuario, sincronizarlo y mantener la flexibilidad de elegir la fuente de lectura en distintos idiomas, sin depender de una sola fuente de información.

El desarrollo se articula en tres componentes principales: un sistema de web scraping encargado de obtener los capítulos desde distintas fuentes, una plataforma web donde el usuario puede gestionar su catálogo personal y su progreso de lectura, y una extensión de navegador que detecta automáticamente el avance de lectura y lo sincroniza con el servidor.

1.1. Motivación

La idea de este proyecto surge a partir de una experiencia personal: el hábito de leer obras en línea y la dificultad para llevar un control consistente del progreso de lectura cuando se utilizan múltiples páginas web.

Ante esta situación, se planteó el desarrollo de una herramienta capaz de centralizar y automatizar ese seguimiento, integrando las capacidades de una plataforma web con una extensión de navegador. Esta solución permitiría conservar el estado de lectura de forma centralizada, sin depender de los servicios ofrecidos por cada fuente y ofreciendo, además, una experiencia más cómoda y unificada para el lector.

1.2. Objetivos

El objetivo general del proyecto es desarrollar una solución que facilite el seguimiento de la lectura de distintas fuentes, de forma automática. En particular, encontramos los siguientes objetivos:

- Desarrollar una plataforma web que permita a los usuarios gestionar su biblioteca personal y consultar su progreso de lectura.

- Implementar una extensión de navegador que detecte automáticamente el capítulo que se está leyendo y sincronice el progreso con el servidor.

- Construir un sistema de web scraping capaz de extraer la lista de capítulos disponibles de múltiples sitios web.

- Diseñar un backend empleando el framework Django (Django Software Foundation, 2025), estructurando la lógica de negocio y la persistencia de los datos.

- Crear una interfaz web funcional utilizando HTML (Mozilla, s.f.), CSS (Mozilla, s.f.) y JavaScript (Mozilla, s.f.).

- Desplegar la aplicación en un entorno accesible al público.

- Fomentar una arquitectura escalable, mantenible y abierta a futuras mejoras.

Estos objetivos permiten alcanzar la competencia principal competencia a desarrollar de este TFG: "Conocimiento y aplicación de las herramientas necesarias para el almacenamiento, procesamiento y acceso a los Sistemas de información, incluidos los basados en web."

1.3. Metodología de desarrollo

Inicialmente, se propuso seguir la metodología ágil Scrum (Kniberg, Skarin, de Mary Poppendieck, & Anderson, 2010), con revisiones periódicas y planificación iterativa. No obstante, a medida que avanzaba el desarrollo, se evidenció que no era posible aplicar Scrum de forma adecuada. La principal limitación fue la dificultad para estimar con precisión el tiempo necesario para cada tarea, debido a la falta de experiencia previa con algunas tecnologías.

Como alternativa, se optó por adoptar un enfoque más flexible, basado en Kanban. Esta metodología permite gestionar el trabajo de forma visual y continua, sin necesidad de establecer sprints ni ciclos fijos. Se organizaron listas de tareas separadas por componente y se estableció un plazo aproximado de un mes de desarrollo para cada uno.

En este contexto, Kanban resultó ser una opción más cómoda y efectiva, ya que facilitó la adaptación al ritmo real del proyecto; permitiendo priorizar tareas según su urgencia y complejidad.

1.4. Estructura del documento

Dado que el sistema propuesto se compone de tres módulos claramente diferenciados, el presente documento se ha estructurado siguiendo esa misma división. Cada componente se describe por separado y, posteriormente, se detalla el proceso de integración

1.5. Estado del arte

El proyecto se sitúa en un ámbito de nicho, sin una solución ampliamente adoptada que defina un estado del arte claro. No obstante, existen diversas aproximaciones que se pueden agrupar en dos grandes categorías: plataformas web y extensiones de navegador.

1.5.1. Plataformas web

Dentro de las plataformas web, existen dos tipos:

- **Con alojamiento de contenido**, como MangaDex, que permite leer obras en múltiples idiomas desde su propia página web y ofrece funciones de seguimiento de lectura.

- **Sin alojamiento de contenido**, como MyAnimeList o Kenmei. MyAnimeList requiere una actualización manual del progreso. Kenmei, más especializado en manga, automatiza la detección de nuevos capítulos y ofrece un sistema de recomendaciones.

1.5.2. Extensiones de navegador

En esta categoría destacan herramientas que actúan sobre lectores externos desde el navegador, sin necesidad de una plataforma web.

- MangaPi: mejora la experiencia de lectura y permite guardar el progreso localmente. No depende de un servidor y ofrece una interfaz de biblioteca en el propio navegador.

- AllMangasReader: permite realizar una búsqueda en tiempo real por título y acceder a los resultados organizados por idioma, lo cual implica tener un servidor dedicado a esta tarea.

1.5.3. Posicionamiento del proyecto

A diferencia de las soluciones anteriores, el presente proyecto combina ambos enfoques: ofrece una plataforma web sin alojamiento del contenido y una extensión de navegador que permite una sincronización automática del progreso de lectura en páginas externas soportadas.

2. Requisitos del sistema

2.1. Scraper

Este sistema tiene como finalidad automatizar la recopilación de información desde diferentes sitios web, para su posterior almacenamiento en una base de datos.

2.1.1. Requisitos generales

ID	Nombre	Descripción
RG-01	Modularidad	El sistema debe estar diseñado para poder dar soporte a nuevos sitios webs, sin tener que modificar la arquitectura principal

Tabla 1: Scraper - Requisitos generales

2.1.2. Requisitos funcionales y no funcionales

ID	Nombre	Descripción
RF-01	Extracción de datos	El sistema debe extraer por cada libro: título principal, títulos auxiliares (si los hubiera), portada, tipo, estado, etiquetas, enlace a la página de detalles y sus capítulos (número, enlace y fecha)
RF-02	Verificación del scanlator	El sistema debe comprobar que el sitio web a scrapear existe en la base de datos
RF-03	Actualizar datos de un libro	El sistema debe poder hacer scraping sobre la página de detalles de un libro para poder actualizarlo sin tener que recurrir a la página principal del scanlator.
RF-04	Verificación de sitio compatible	El sistema debe poder verificar si un sitio web es compatible con el scraping, esto es, que no implemente protección de robots como la de Cloudflare.
RF-05	Detección de datos corruptos	Si falta información obligatoria, el sistema debe lanzar una excepción para evitar almacenar datos incompletos.
RF-06	Gestión de reintentos	El sistema debe poder reintentar el scraping en caso de detectar errores.
RF-07	Almacenamiento	El sistema debe almacenar libros y capítulos en la base de datos relacional.
RF-08	Registro de logs	Se deben registrar los errores en caso de agotar las oportunidades que ofrece el sistema de reintentos.

Tabla 2: Scraper - Requisitos funcionales

ID	Nombre	Descripción
RNF-01	Trazabilidad de errores	Los logs deben contener información clara sobre el origen y contexto de cada error.
RNF-02	Soporte para sitios dinámicos	El sistema debe poder renderizar contenido JavaScript durante el scraping.

Tabla 3: Scraper - Requisitos no funcionales

2.1.3. Actores

ID	Nombre	Descripción
A-01	Administrador	Persona encargada de ejecutar los scripts de scraping y supervisar los resultados

Tabla 4: Scraper - Actores

2.1.4. Casos de uso

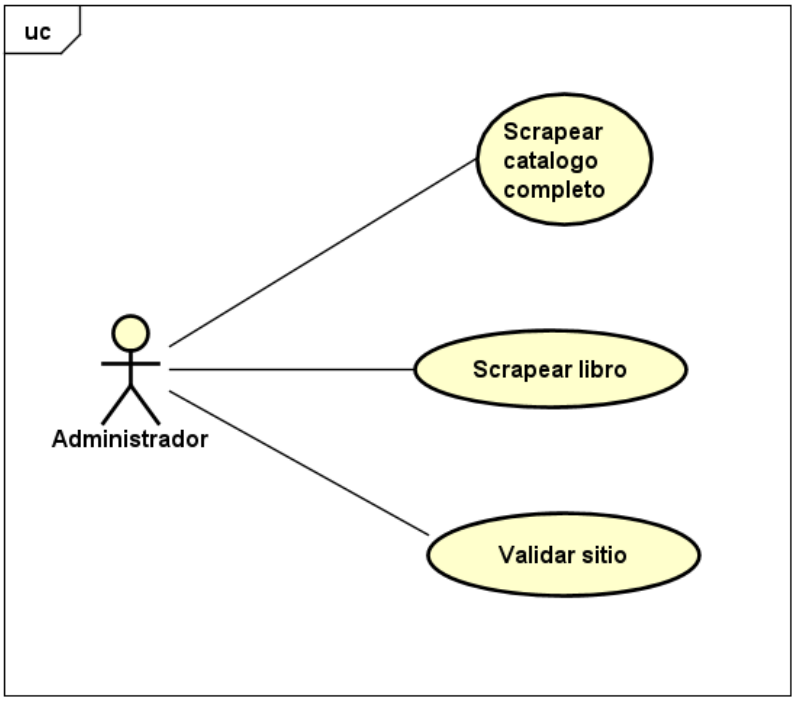

Ilustración 1: Scraper - Diagrama de casos de uso (Administrador)

CU-01	**Scrapear catálogo completo**
Precondición	- La página web a scrapear, debe estar registrada en la base de datos y estar soportada por el sistema de scraping.
Dependencias	A-01 (Administrador) RF-01 (Extracción de datos) RF-02 (Verificación del scanlator) RF-05 (Detección de datos corruptos) RF-06 (Gestión de reintentos) RF-07 (Almacenamiento) RF-08 (Registro de logs)

Descripción	Extrae todos los libros de catálogo del sitio web a scrapear y actualiza la base de datos con dicha información
Secuencia normal	

PASO	ACCIÓN
1	Verificar que el sitio web a scrapear está registrado en la base de datos.
2	Iniciar la extracción desde la página principal del sitio web.
3	Por cada libro mostrado en la página actual, se extraen las urls necesarias para aplicar un scraping específico sobre la página de detalles de cada uno.
3.1	Sobre la página de detalles de cada libro se vuelve a hacer scraping para obtener el resto de datos de cada libro, incluyendo sus capítulos.
4	Se verifica la integridad de los datos extraídos de cada libro, esto es, que no hayan campos obligatorios en blanco.
5	Si los datos son válidos, se guardan en la base de datos.
6	Si existe una siguiente página en el sitio web (paginación), se repite el proceso desde el paso 2.

Postcondición	Se agregan y/o actualizan todos los libros, de la página web objetivo, en la base de datos.
Excepciones	

PASO	ACCIÓN
1.A	Si el sitio no está registrado en la base de datos, se lanza un error y se detiene el proceso.
4.A	Si hay datos obligatorios que no se han podido scrapear, se registra como un error y pasa al sistema de reintentos.
X	Si durante el proceso de scraping, para sitios que requieran cargar contenido dinámico (JavaScript) ocurre un timeout, se redirige la petición al sistema de reintentos para volver a tratar de scrapear dicho sitio web.

	Y	Si una página web agota el máximo de reintentos, se deja de derivar al sistema de reintentos y se plasma el fallo en un archivo de logs de error concreto, para su posterior tratamiento.
Comentarios	- La lógica de paginación debe permitir scrapear sitios con múltiples páginas de catálogo de libros. - Los datos a scrapear, por cada libro, son: título, imagen de la portada, url de la página de detalles, estado, capítulos (esto incluye el número, fecha de publicación y enlace al capítulo), lista de otros títulos (suelen ser traducciones en otros idiomas), tipo, lista de géneros. - Los datos obligatorios de un libro, para no ser considerado como "corrupto" son: título, imagen de la portada, url de la página de detalles. Además, debe tener, al menos, un capítulo con: número del capítulo y url de enlace al capítulo. - Si se detecta un error de integridad por datos incompletos o corruptos, se genera un mensaje de error indicando los campos faltantes (obligatorios o no).	

Tabla 5: Scraper - CU-01 (Scrapear catálogo completo)

CU-02	**Scrapear libro**
Precondición	- La página web a scrapear, debe estar registrada en la base de datos y estar soportada por el sistema de scraping.
Dependencias	A-01 (Administrador) RF-01 (Extracción de datos) RF-02 (Verificación del scanlator) RF-03 (Actualizar datos de un libro) RF-05 (Detección de datos corruptos) RF-06 (Gestión de reintentos) RF-07 (Almacenamiento) RF-08 (Registro de logs)

Descripción	Permite extraer los datos de un libro concreto perteneciente a una página web concreta y guardarlos o actualizarlos en la base de datos.
Secuencia normal	

PASO	ACCIÓN
1	Verificar que el sitio web a scrapear está registrado en la base de datos.
2	Se extraen todos los datos (del libro, incluyendo sus capítulos) de la página de detalles proporcionada.
3	Se verifica la integridad de los datos extraídos de cada libro (campos obligatorios).
4	Si los datos son válidos, se guardan en la base de datos.

Postcondición	El libro es actualizado o agregado a la base de datos del sistema.
Excepciones	

PASO	ACCIÓN
1.A	Si el sitio no está registrado en la base de datos, se lanza un error y se detiene el proceso.
3.A	Si hay datos obligatorios que no se han podido scrapear, se registra como un error y pasa al sistema de reintentos.
X	Si durante el proceso de scraping, para sitios que requieran cargar contenido dinámico (JavaScript) ocurre un timeout, se redirige la petición al sistema de reintentos para volver a tratar de scrapear dicho sitio web
Y	Si la página web agota el máximo de reintentos, se deja de derivar al sistema de reintentos y se plasma su error en un archivo de logs de error concreto, para su posterior tratamiento.

Comentarios	- Los datos a scrapear, por cada libro, son: título, imagen de la portada, url de la página de detalles, estado, capítulos (esto incluye el número, fecha de publicación y enlace al capítulo), lista de otros títulos (suelen ser traducciones en otros idiomas), tipo, lista de géneros.

	- Los datos obligatorios de un libro, para no ser considerado como “corrupto” son: título, imagen de la portada, url de la página de detalles. Además, debe tener, al menos, un capítulo con: número del capítulo y url de enlace al capítulo. - Si se detecta un error de integridad por datos incompletos o corruptos, se genera un mensaje de error indicando los campos faltantes (obligatorios o no).

Tabla 6: Scraper - CU-02 (Scrapear libro)

CU-03	**Validar sitio**
Precondición	- La página web a scrapear, debe estar registrada en la base de datos y estar soportada por el sistema de scraping.
Dependencias	A-01 (Administrador) RF-04 (Verificación de sitio compatible)
Descripción	Permite verificar la viabilidad de una página web, antes de darle soporte. Para ello se renderiza la página para comprobar que no exista ningún bloqueo de scraping (como el de Cloudflare).
Secuencia normal	**PASO** **ACCIÓN** **1** Renderiza la página (de la url proporcionada por parámetros).
Comentarios	- Se trata de un paso previo para dar soporte a una nueva página web.

Tabla 7: Scraper - CU-03 (Validar sitio)

2.2. Página web

2.2.1. Requisitos generales

ID	Nombre	Descripción
RG-01	Plataforma centralizada	El sistema debe ofrece una plataforma web para el seguimiento, visualización y lectura de libros, de múltiples fuentes.

Tabla 8: Página web - Requisitos generales

2.2.2. Requisitos funcionales y no funcionales

ID	Nombre	Descripción
RF-01	Inicio de sesión	Permitir autenticarse en el sistema mediante nombre de usuario y contraseña.
RF-02	Registro de usuario	Permitir a nuevos usuarios registrarse mediante un formulario.
RF-03	Cierre de sesión	Permitir al usuario cerrar sesión.
RF-04	Visualización de libros	Mostrar en la página principal todos los libros del catálogo.
RF-05	Búsqueda por título	Permitir realizar búsquedas de libros por título.
RF-06	Búsqueda avanzada	Permitir aplicar filtros de búsqueda.
RF-07	Vista de detalles del libro	Poder ver toda la información relevante del libro seleccionado.
RF-08	Mostrar todas las fuentes de cada capítulo	Poder visualizar, para cada capítulo de un libro, todos los enlaces disponibles de distintas fuentes (scanlators). Estos enlaces estarán agrupados por capítulo y no por fuente.
RF-09	Estado de lectura	Permitir al usuario marcar su estado con respecto a un libro (“Reading”, “On Hold” o “Dropped”).
RF-10	Favoritos	Permitir añadir o quitar libros de favoritos.

RF-11	Marcar capítulo como leído	Permitir al usuario marcar manualmente (o automáticamente) el último capítulo leído.
RF-12	Configuración de usuario	Permitir modificar al usuario la configuración de su cuenta: imagen de avatar, lista de idiomas preferidos y activar/desactivar notificaciones por correo.
RF-13	Ver lista de libros seguidos (MyList)	Permitir al usuario ver todos los libros seguidos, pulsando un botón.
RF-14	Continuar lectura	Redirigir al usuario al siguiente capítulo no leído, en base a sus idiomas preferidos.
RF-15	Notificaciones temporales	Mostrar mensajes emergentes temporales tras acciones del usuario.
RF-16	Fusión de libros	Permitir al administrador fusionar libros (Book) y/o wrappers (BookWrapper).
RF-17	Selección de atributos al fusionar libros	Permitir al administrador seleccionar qué atributos hereda el wrapper (BookWrapper) resultante.

Tabla 9: Página web - Requisitos funcionales

ID	Nombre	Descripción
RNF-01	Separación de lógica	La lógica de presentación y la lógica de negocio deben estar claramente separadas
RNF-02	Cifrado de contraseñas	Las contraseñas deben almacenarse en la base de datos de forma cifrada.
RNF-03	Protección contra CSRF	El sistema debe incluir tokens CSRF en los formularios para prevenir ataques de falsificación de peticiones.
RNF-04	Protección contra inyección SQL	El sistema debe incluir protección contra inyecciones SQL

RNF-05	Acceso por HTTPS	La plataforma debe funcionar exclusivamente a través del protocolo HTTPS en producción.
RNF-06	Usabilidad	La interfaz debe ser clara e intuitiva, orientada a una navegación fluida y sin necesidad de aprendizaje previo.
RNF-07	Navegación fluida	La interfaz debe permitir una interacción fluida mediante el uso de peticiones asíncronas, evitando cargas completas de página.
RNF-08	Información sobre cookies	El sistema debe informar al usuario sobre el uso de cookies mediante un mensaje visible en el pie de página.

Tabla 10: Página web - Requisitos no funcionales

2.2.3. Actores

ID	Nombre	Descripción
A-01	Usuario no logueado	Actor que accede al sistema sin iniciar sesión. Solo puede consultar información.
A-02	Usuario logueado	Actor autenticado en el sistema. Puede consultar e interactuar con el sistema (puede acceder a las operaciones de un usuario no logueado).
A-03	Administrador web	Actor con acceso al panel de administración (también puede consultar e interactuar con el sistema como cualquier otro usuario logueado).

Tabla 11: Página web - Actores

2.2.4. Casos de uso

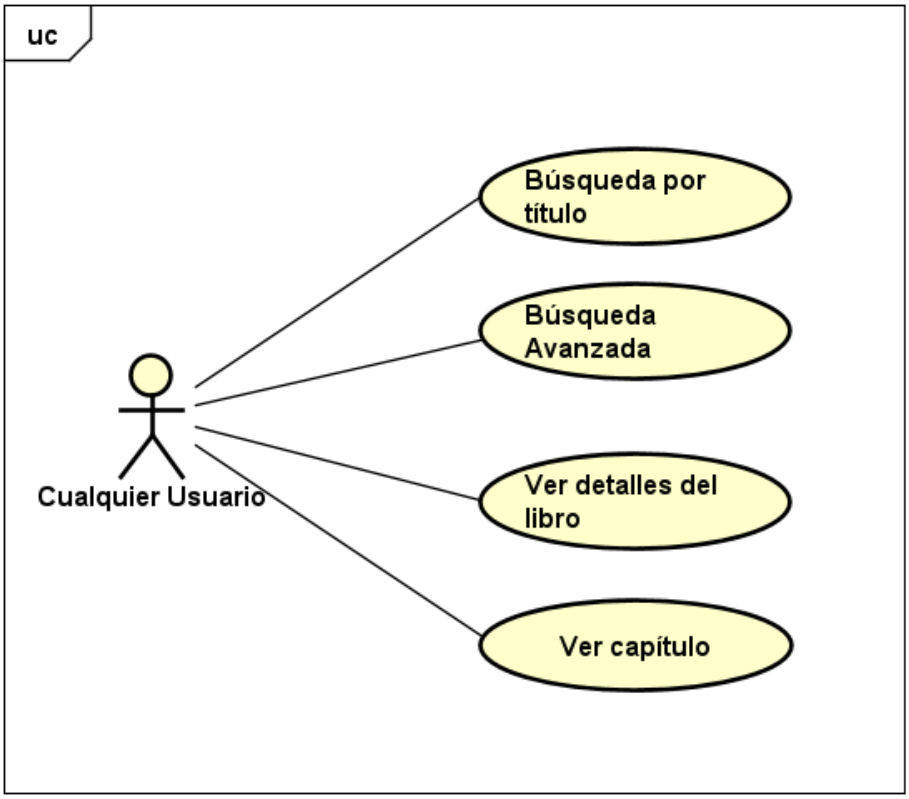

Ilustración 2: Página web - Diagrama de casos de uso (cualquier usuario)

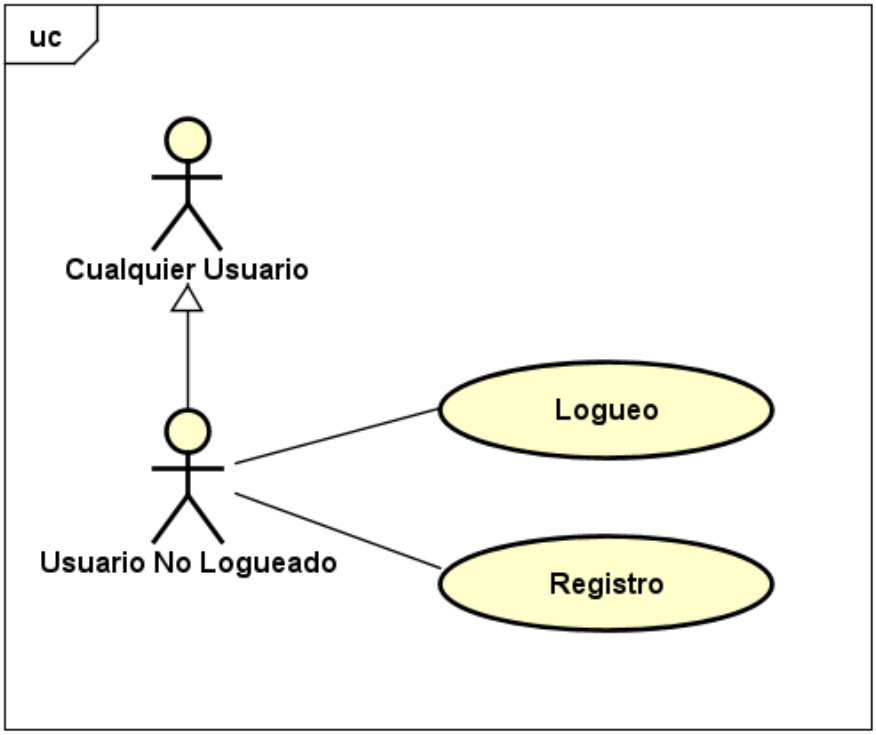

Ilustración 3: Página web - Diagrama de casos de uso (Usuario No Logueado)

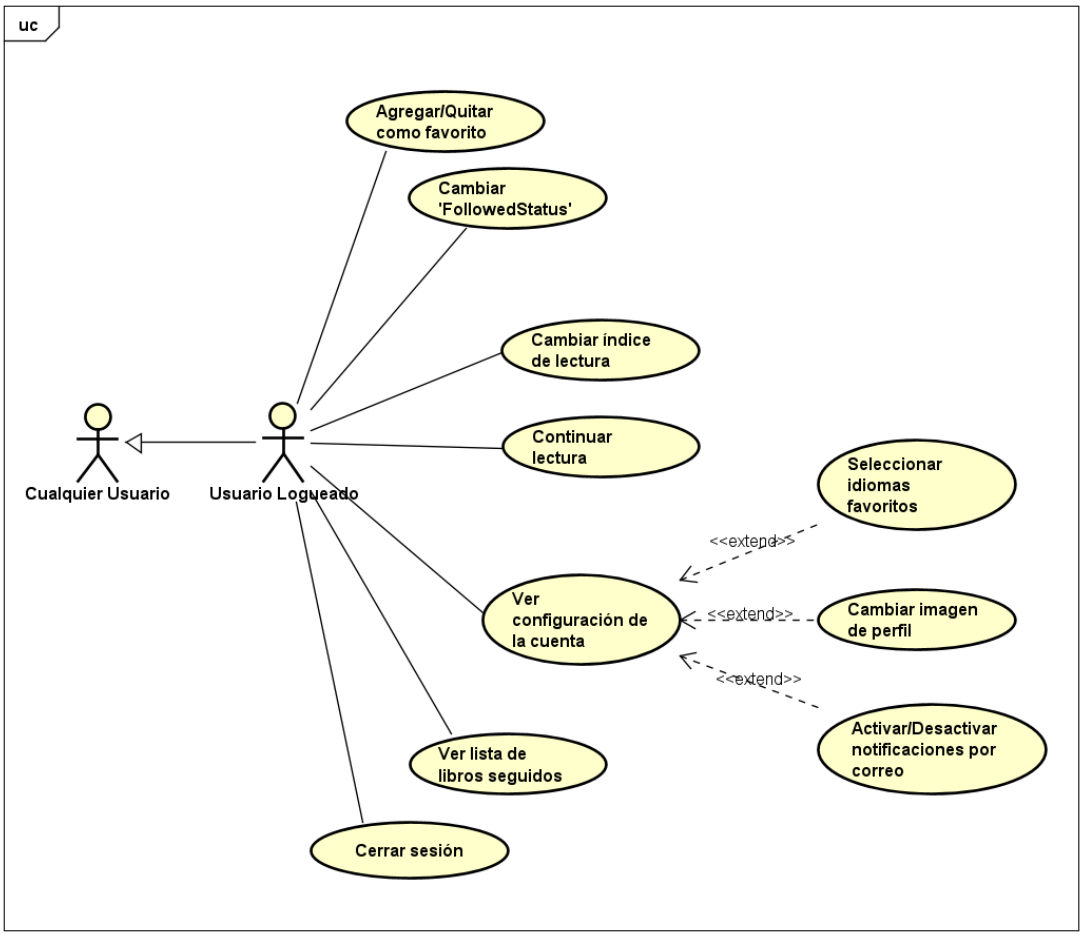

Ilustración 4: Página web - Diagrama de casos de uso (Usuario Logueado)

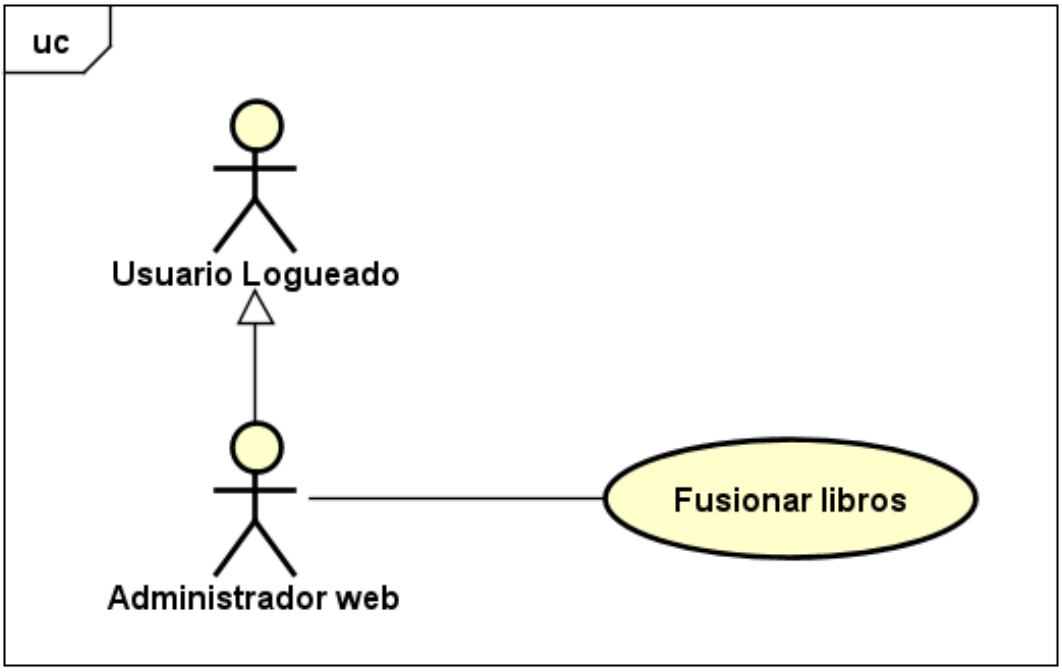

Ilustración 5: Página web - Diagrama de casos de uso (Administrador web)

CU-01	**Logueo**
Precondición	- Tener una cuenta de usuario - No estar logueado en el sistema.
Dependencias	A-01 (Usuario no logueado) RF-01 (Inicio de sesión)
Descripción	Permite loguearte en la aplicación con tu cuenta de usuario.
Secuencia normal	**PASO** / **ACCIÓN** **1** El sistema pide introducir: nombre de usuario y contraseña. **2** El usuario introduce los datos **3** El sistema valida los datos
Postcondición	El usuario se queda logueado y se produce una redirección a la página principal.
Excepciones	**PASO** / **ACCIÓN** **3.A** Si alguno de los campos está vacío, aparece un mensaje indicando que es obligatorio. **3.B** Si el par nombre de usuario y contraseña no son correctos, aparece un mensaje de error indicando el fallo.
Comentarios	- Desde esta vista, se puede acceder a la vista de registro y a la página principal.

Tabla 12: Página web - CU-01 (Logueo)

CU-02	Registro
Precondición	- No estar logueado en el sistema.
Dependencias	A-01 (Usuario no logueado) RF-02 (Registro de usuario)
Descripción	Permite crear una nueva cuenta en el sistema.
Secuencia normal	**PASO** **ACCIÓN** **1** El sistema pide introducir: nombre de usuario, correo electrónico y contraseña. Y aceptar los términos y condiciones. **2** El usuario introduce los datos y acepta los términos y condiciones. **3** El sistema valida los datos
Postcondición	El usuario se queda logueado y se produce una redirección a la página principal.
Excepciones	**PASO** **ACCIÓN** **3.A** Si alguno de los campos obligatorios está vacío, aparece un mensaje indicando su obligatoriedad. Lo mismo ocurre en caso de no haber aceptado los términos y condiciones. **3.B** Si ya existe una cuenta con ese correo electrónico, aparece un mensaje indicando el error. **3.C** Si ya existe una cuenta con ese nombre de usuario, aparece un mensaje indicando el error.
Comentarios	- Desde esta vista, se puede acceder a la vista del Login y a la página principal

Tabla 13: Página web - CU-02 (Registro)

CU-03	Búsqueda por título
Precondición	
Dependencias	A-01 (Usuario no logueado) A-02 (Usuario logueado) A-03 (Administrador web) RF-05 (Búsqueda por título)
Descripción	Permite hacer una búsqueda por título, sobre la biblioteca de libros del sistema.
Secuencia normal	**PASO** — **ACCIÓN** **1** — El usuario introduce un título completo o una parte de él. **2** — El sistema buscará todos los libros que contengan ese título o parte de él. **3** — El sistema muestra todos los resultados encontrados.
Postcondición	
Excepciones	**PASO** — **ACCIÓN** **3.A** — En caso de no haber resultados, aparece un mensaje indicándolo.

Tabla 14: Página web - CU-03 (Búsqueda por título)

CU-04	Búsqueda Avanzada
Precondición	
Dependencias	A-01 (Usuario no logueado) A-02 (Usuario logueado) A-03 (Administrador web) RF-06 (Búsqueda avanzada)
Descripción	Permite aplicar filtros sobre la búsqueda de libros.
Secuencia normal	**PASO** — **ACCIÓN** **1** — El usuario selecciona los filtros que quiera aplicar. **2** — El sistema hace una búsqueda por título (si lo hubiera), aplicando dichos filtros. **3** — El sistema muestra los resultados de la búsqueda.
Postcondición	
Excepciones	**PASO** — **ACCIÓN** **3.A** — En caso de no haber resultados, aparece un mensaje indicándolo.
Comentarios	Los filtros disponibles son los siguientes: - Orden: por título, por número de capítulos, por número de favoritos, por número de personas que lo siguen o por fecha de subida. - Dirección: ascendente o descendente. - Género o Tag (con inclusión y exclusión)

	- Estado: activo, hiatus o finalizado. - Tipo de obra: manga, manhua o manhwa. - Estado de seguimiento: leyendo, pausado, descartado. El filtro de ‘estado de seguimiento’ solo se aplica para usuarios logueados.

Tabla 15: Página web - CU-04 (Búsqueda avanzada)

CU-05	**Ver detalles del libro**
Precondición	
Dependencias	A-01 (Usuario no logueado) A-02 (Usuario logueado) A-03 (Administrador web) RF-07 (Vista de detalles del libro) RF-08 (Mostrar todas las fuentes de cada capítulo)
Descripción	Muestra más información de cada libro.
Secuencia normal	**PASO** / **ACCIÓN** **1** / El usuario hace clic en la portada de un libro. **2** / El sistema muestra todos los datos del libro seleccionado.
Postcondición	
Excepciones	**PASO** / **ACCIÓN** **2.A** / Si el ID del libro seleccionado no existe, se muestra un error 404.

Tabla 16: Página web - CU-05 (Ver detalles del libro)

CU-06	Ver capítulo
Precondición	Estar en la página de detalles de un libro que tenga capítulos.
Dependencias	A-01 (Usuario no logueado) A-02 (Usuario logueado) A-03 (Administrador web) RF-11 (Marcar capítulo como leído)
Descripción	Permite acceder a la página de lectura (externa) del capítulo del libro actual.
Secuencia normal	**PASO** — **ACCIÓN** **1** — El usuario despliega la lista de capítulos disponibles **2** — El usuario selecciona una fuente de las ofrecidas para dicho capítulo. **3** — El sistema abre en una nueva pestaña la URL del capítulo seleccionado. **4** — Si el usuario está logueado, se guarda como último capítulo leído.
Postcondición	Se abre en una nueva pestaña del navegador, la página de lectura del capítulo seleccionado. Se guarda como capítulo leído en caso de estar logueado.
Comentarios	- Leer un capítulo no modifica el 'FollowedStatus' del usuario con respecto a dicho libro. Esto implica que si, por ejemplo el libro estaba en 'Dropped', leer uno de sus capítulos no lo pone en 'Reading'. Esta decisión de diseño se toma para evitar situaciones inesperadas para el usuario.

Tabla 17: Página web - CU-06 (Ver capítulo)

CU-07	Agregar/Quitar como favorito
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado)

	A-03 (Administrador web) RF-10 (Favoritos) RF-15 (Notificaciones temporales)
Descripción	Permite marcar o desmarcar un libro como favorito
Secuencia normal	**PASO** / **ACCIÓN** **1** El usuario hace clic en el botón de favoritos. **2** El sistema verifica si ya está en favoritos: en caso de estarlo lo borra de favoritos y en caso contrario lo agrega a favoritos **3** Aparece un mensaje temporal confirmando que el libro ha sido borrado o agregado a favoritos.
Postcondición	El libro se guarda o borra de favoritos

Tabla 18: Página web - CU-07 (Agregar/Quitar como favorito)

CU-08	**Cambiar 'FollowedStatus'**
Precondición	- Estar logueado en el sistema
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-09 (Estado de lectura) RF-15 (Notificaciones temporales)
Descripción	Permite cambiar la relación del usuario con un libro a los modos: "Reading", "On Hold" o "Dropped".

Secuencia normal	<table><tr><th>PASO</th><th>ACCIÓN</th></tr><tr><td>1</td><td>El usuario selecciona a que estado cambiar.</td></tr><tr><td>2</td><td>El sistema actualiza el estado seleccionado.</td></tr><tr><td>3</td><td>Aparece un mensaje temporal informando al usuario del nuevo estado seleccionado.</td></tr></table>
Postcondición	Se cambia el ‘FollowedStatus’ del libro.

Tabla 19: Página web - CU-08 (Cambiar 'FollowedStatus')

CU-09	**Cambiar índice de lectura**
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-11 (Marcar capítulo como leído) RF-15 (Notificaciones temporales)
Descripción	Permite cambiar, de forma manual, el último capítulo leído de un libro.
Secuencia normal	

	PASO **ACCIÓN** **1** El usuario selecciona un capítulo como “último leído“. **2** El sistema actualiza los datos. **3** Aparece un mensaje temporal informando al usuario del cambio.
Postcondición	Se actualiza el índice del último capítulo leído de dicho libro.
Excepciones	**PASO** **ACCIÓN** **2.A** En caso de no estar logueado, el sistema muestra un mensaje temporal indicando que dicha opción solo está disponible para usuarios logueados. **2.B** En caso de no seguir el libro (haberlo agregado a favoritos, o tener un ‘FollowedStatus’), el sistema muestra un mensaje temporal indicando que es necesario seguirlo para modificar el índice de lectura del libro.

Tabla 20: Página web - CU-09 (Cambiar índice de lectura)

CU-10	**Continuar lectura**
Precondición	- Estar logueado en el sistema. - Seguir dicho libro (agregándolo a favoritos o agregándole un ‘FollowedStatus’).
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-11 (Marcar capítulo como leído) RF-14 (Continuar lectura)

	RF-15 (Notificaciones temporales)
Descripción	Permite continuar la lectura de un libro, abriendo el siguiente capítulo en una nueva pestaña del navegador.
Secuencia normal	**PASO** / **ACCIÓN** **1** El usuario hace clic en el botón para continuar la lectura (botón 'Next Chapter'). **2** El sistema busca el siguiente capítulo disponible, **3** Se filtran los enlaces del capítulo según los idiomas preferidos. **4** Se abre el capítulo en una nueva pestaña del navegador. **5** El sistema actualiza el índice de lectura de libro al índice del nuevo capítulo abierto.
Postcondición	Se abre en una nueva pestaña del navegador la página de lectura del siguiente capítulo y el sistema actualiza el índice de lectura del libro.
Excepciones	**PASO** / **ACCIÓN** **2.A** Si el usuario no está siguiendo el libro, se muestra un mensaje temporal indicándole que necesita seguirlo. **2.B** Si no hay siguiente capítulo, se muestra un mensaje temporal indicando al usuario que ya no hay más capítulos por leer. **3.A** Si no hay enlaces en ninguno de los idiomas preferidos, se muestra un mensaje temporal al usuario indicándole el fallo e instándole a hacerlo de forma manual.

Tabla 21: Página web - CU-10 (Continuar lectura)

CU-11	Ver configuración de la cuenta
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-12 (Configuración de usuario)
Descripción	Permite al usuario acceder y modificar las preferencias de su cuenta.
Secuencia normal	**PASO** / **ACCIÓN** **1** El usuario hace clic en el botón para ver la configuración de su cuenta. **2** El sistema muestra la vista de configuración de su cuenta.

Tabla 22: Página web – CU-11 (Ver configuración de la cuenta)

CU-12	Seleccionar idiomas favoritos
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-12 (Configuración de usuario) RF-15 (Notificaciones temporales) CU-11 (Ver configuración de la cuenta)
Descripción	Permite agregar o quitar idiomas a la lista de idiomas preferidos.
Secuencia normal	

	PASO	**ACCIÓN**
	1	El usuario selecciona aquellos idiomas preferidos.
	2	El usuario hace clic en el botón de guardar cambios.
	3	El sistema actualiza sus preferencias.
	4	Se muestra un mensaje temporal, indicándole al usuario que sus preferencias han sido actualizadas.
Postcondición	Se actualiza la lista de idiomas preferidos	

Tabla 23: Página web – CU-12 (Seleccionar idiomas favoritos)

CU-13	**Cambiar imagen de perfil**
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-12 (Configuración de usuario) RF-15 (Notificaciones temporales) CU-11 (Ver configuración de la cuenta)
Descripción	Permite escoger una imagen de perfil (de un catálogo reducido).
Secuencia normal	

	PASO **ACCIÓN** **1** El usuario selecciona una imagen de perfil. **2** El usuario hace clic en el botón de guardar cambios. **3** El sistema actualiza sus preferencias. **4** Se muestra un mensaje temporal, indicándole al usuario que sus preferencias han sido actualizadas.
Postcondición	Se actualiza la imagen de perfil.

Tabla 24: Página web – CU-13 (Cambiar imagen de perfil)

CU-14	**Activar/Desactivar notificaciones por correo**
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-12 (Configuración de usuario) RF-15 (Notificaciones temporales) CU-11 (Ver configuración de la cuenta)
Descripción	Permite activar o desactivar las notificaciones por correo.
Secuencia normal	

	PASO **ACCIÓN** **1** El usuario hace clic para activar o desactivar las notificaciones por correo. **2** El usuario hace clic en el botón de guardar cambios. **3** El sistema actualiza sus preferencias. **4** Se muestra un mensaje temporal, indicándole al usuario que sus preferencias han sido actualizadas.
Postcondición	Se activan o desactivan las notificaciones por correo.

Tabla 25: Página web – CU-14 (Activar/Desactivar notificaciones por correo)

CU-15	**Ver lista de libros seguidos**
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-13 (Ver lista de libros seguidos) RF-15 (Notificaciones temporales) CU-04 (Búsqueda Avanzada)
Descripción	Hace una búsqueda avanzada, mostrando los libros marcados como seguidos.
Secuencia normal	**PASO** **ACCIÓN** **1** El usuario hace clic en el botón para ver su lista de libros seguidos. **2** El sistema muestra los libros seguidos, aplicando una búsqueda avanzada.

Postcondición	Se muestran los libros seguidos por el usuario.
Excepciones	**PASO** **ACCIÓN** **2.A** En caso de no haber resultados, aparece un mensaje indicándolo.

Tabla 26: Página web – CU-15 (Ver lista de libros seguidos)

CU-16	**Cerrar sesión**
Precondición	- Estar logueado en el sistema.
Dependencias	A-02 (Usuario logueado) A-03 (Administrador web) RF-03 (Cierre de sesión)
Descripción	Permite cerrar la sesión de usuario.
Secuencia normal	**PASO** **ACCIÓN** **1** El usuario hace clic en el botón de cierre de sesión **2** El sistema cierra la sesión y hace una redirección a la página principal.
Postcondición	El usuario pasa a ser considerado como usuario no logueado y es redirigido a la página principal.

Tabla 27: Página web – CU-16 (Cerrar sesión)

CU-17	Fusionar libros
Precondición	- Haber iniciado sesión como administrador.
Dependencias	A-03 (Administrador web) RF-16 (Fusión de libros) RF-17 (Selección de atributos al fusionar libros)
Descripción	Permite fusionar libros, obtenidos de distintas fuentes (scanlators) en una única entidad: BookWrapper.
Secuencia normal	

PASO	ACCIÓN
1	El usuario filtra los libros por título y selecciona aquellos que desee fusionar
1.1	El sistema añade los libros seleccionados a la lista de libros a fusionar.
2	El usuario filtra los BookWrappers por título y selecciona aquellos que desea agregar a la fusión.
2.1	El sistema añade todos los libros contenidos en los wrappers seleccionados a la lista de libros a fusionar.
3	El usuario puede eliminar libros de la lista de fusión.
4	El usuario hace clic en el botón de “Generar opciones”
5	El sistema genera una interfaz para seleccionar los atributos del nuevo BookWrapper (titulo, imagen, estado, tipo y sinopsis). Por defecto se cargan los del primer libro.
6	El usuario selecciona los atributos que heredará el wrapper resultante.
7	El usuario puede volver a los pasos 1, 2 y 3, pero debe repetir el paso 4 para aplicar los cambios.
8	El usuario hace clic en el botón de “Generar fusión”, para confirmar la fusión.

	9	El sistema genera la fusión y crea un nuevo BookWrapper
Postcondición	Se crea un BookWrapper que agrupa todos los libros (Books) seleccionados.	
Comentarios	- Un Book es el resultado de scrapear un libro. - Un BookWrapper agrupa uno o más Books, que representan el mismo contenido desde distintas fuentes - Esta funcionalidad ayuda a evitar duplicados y mantener una estructura unificada en el sistema. - Los BookWrappers son los únicos objetos visibles en la interfaz de usuario final.	

Tabla 28: Página web - CU-17 (Fusionar libros)

2.3. Extensión de navegador

2.3.1. Requisitos generales

ID	Nombre	Descripción
RG-01	Compatibilidad	La extensión deber funcionar en navegadores basados en Chromium (Chrome, Edge, Brave, etc.).
RG-02	Modularidad	La arquitectura debe permitir dar soporte a nuevos sitios web sin modificar su núcleo funcional.

Tabla 29: Extensión de navegador - Requisitos generales

2.3.2. Requisitos funcionales y no funcionales

ID	Nombre	Descripción
RF-01	Detección del capítulo	La extensión debe registrar localmente el capítulo como leído en el almacenamiento interno.
RF-02	Visibilidad de sitio soportado	Debe mostrar visualmente si la página actual pertenece a un sitio web soportado por la extensión.

RF-03	Notificaciones	Debe poder mostrar notificaciones para informar al usuario sobre el éxito o fallo de eventos importantes.
RF-04	Sincronización manual	Debe ofrecer la opción de sincronizar manualmente el progreso de lectura contra el servidor, únicamente cuando el usuario se encuentre en una página web del sistema.
RF-05	Sincronización automática	Debe sincronizar automáticamente el historial de lectura del usuario cuando se detecte que está en una página perteneciente al sistema..
RF-06	Paso de capítulo	Debe permitir al usuario pasar de capítulo usando el teclado, siempre que sea posible.
RF-07	Soporte inteligente	La extensión debe ser capaz de funcionar en los múltiples sitios web soportados sin requerir configuración adicional, siendo transparente para el usuario.
RF-08	Visualización de capítulos por sincronizar	La extensión debe permitir al usuario visualizar los capítulos registrados (como leídos) localmente que aún no han sido sincronizados con el servidor.

Tabla 30: Extensión de navegador - Requisitos funcionales

ID	Nombre	Descripción
RNF-01	Persistencia local	La extensión debe poder almacenar de forma persistente el historial de lectura.
RNF-02	Seguridad de la comunicación	Las peticiones realizadas por la extensión hacia el backend deben incluir un token CSRF válido, extraído del entorno del usuario, con el fin de proteger contra ataques de falsificación entre sitios (CSRF) y garantizar la autenticidad de las operaciones.
RNF-03	Privacidad	La extensión no debe recopilar ni transmitir información personal del usuario, más allá de la necesaria para su funcionamiento básico.
RNF-09	Internacionalización	La extensión debe mostrar su interfaz en el idioma de navegación del usuario, siempre que exista una traducción

		disponible. Como mínimo, se deben proporcionar versiones en español e inglés.

Tabla 31: Extensión de navegador - Requisitos no funcionales

2.3.3. Actores

ID	Nombre	Descripción
A-01	Usuario (extensión)	Persona que usa la extensión para llevar un seguimiento de su historial de lectura.
A-02	Backend (servidor)	Servidor externo que recibe, valida y almacena el historial de lectura del usuario.
A-03	Background	Componente interno de la extensión que opera de forma autónoma detectando eventos, coordinando procesos y que se comunica con el servidor sin intervención directa del usuario.

Tabla 32: Extensión de navegador - Actores

2.3.4. Casos de uso

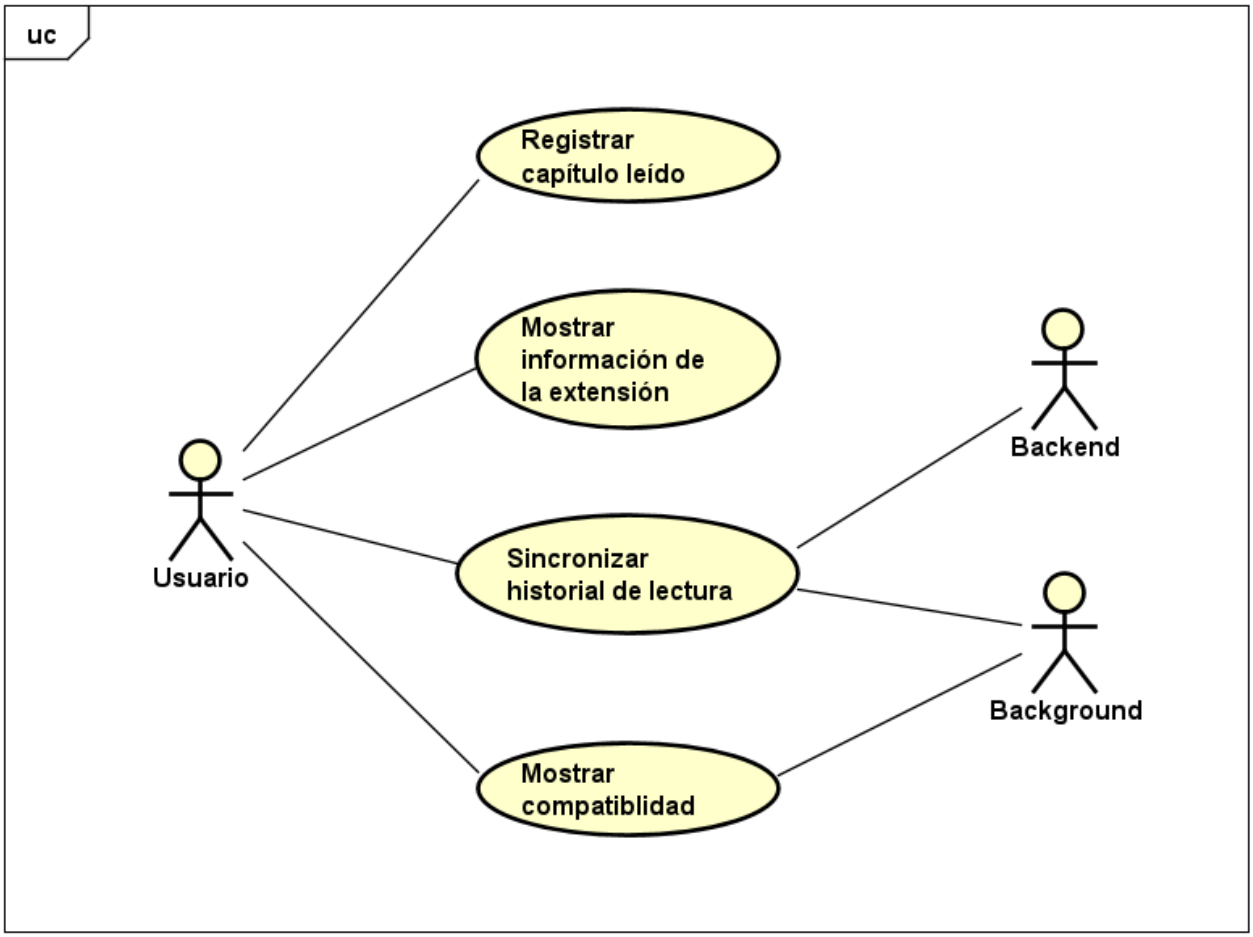

Ilustración 6: Extensión de navegador - Diagrama de casos de uso

CU-01	**Registrar capítulo leído (pasar de capítulo)**
Precondición	- El usuario está navegando por un sitio web soportado por la extensión - El usuario se encuentra en la página de lectura de un capítulo
Dependencias	A-01 (Usuario) RF-01 (Detección del capítulo) RF-03 (Notificaciones) RF-06 (Paso de capítulo)

	RF-07 (Soporte inteligente)
Descripción	Cuando el usuario navega al siguiente capítulo (ya sea mediante las flechas del teclado o haciendo clic en el botón de siguiente), la extensión registra localmente el capítulo actual (antes de cambia de capítulo) como leído. Este proceso guarda en el almacenamiento interno: el título, número y origen del capítulo (web soportada que lo hospeda).
Secuencia normal	**PASO** / **ACCIÓN** **1** El usuario pasa de capítulo; hace clic en el botón “Siguiente” o pulsa la flecha del teclado ‘→’. **2** La extensión detecta el cambio y guarda los datos del capítulo actual como leído. **3** Se notifica al usuario que el capítulo ha sido marcado como leído.
Postcondición	El capítulo actual queda registrado como leído en el almacenamiento local.
Excepciones	**PASO** / **ACCIÓN** **2.A** Si la extracción de los datos del capítulo actual falla (por cambios en la estructura del sitio o errores del DOM), no se guarda ningún dato y se omite el registro.

Tabla 33: Extensión de navegador - CU-01 (Registrar capítulo leído)

CU-02	**Sincronizar historial de lectura**
Precondición	- El usuario debe haber iniciado sesión en la página web principal y encontrase en ésta.
Dependencias	A-01 (Usuario) A-02 (Backend) A-03 (Background) RF-03 (Notificaciones)

	RF-04 (Sincronización manual) RF-05 (Sincronización automática) CU-04 (Consultar estado de la extensión)
Descripción	Sincroniza el listado de últimos capítulos leídos contra el servidor
Secuencia normal	

PASO	ACCIÓN
0.A	El usuario hace clic en el icono de la extensión, abre el popup y selecciona el botón de sincronización manual
0.B	Alternativamente, el usuario navega o cambia a una pestaña correspondiente al dominio de la página web principal.
1	El background verifica que hayan datos de lectura (capítulos leídos) por sincronizar.
2	El background intenta obtener el token CSRF de la página actual.
3	El background envía una petición POST con los datos (el historial de lectura guardado localmente y el token CSRF), al servidor
4	El servidor valida la autenticidad del usuario y procesa cada entrada del historial de lectura recibido:
4.1	Si el libro ya estaba siendo seguido por el usuario, actualiza el número del capítulo leído.
4.2	Si el libro no estaba siendo seguido, se marca como seguido y se registra el capítulo recibido como el último leído.
5	El servidor responde a la extensión con la lista de libros que si se han sincronizado correctamente.
6	El background elimina del almacenamiento local a aquellos libros sincronizados con éxito.
7	Se muestra una notificación al usuario con el resultado de la operación (una lista de los libros actualizados).

Postcondición	Los capítulos leídos que se han sincronizado contra el servidor se borran del almacenamiento local del usuario.
Excepciones	**PASO** **ACCIÓN** **1.A** Si no hay capítulos, se para el flujo. **2.A** Si no hay token CSRF, se para el flujo; se muestra un warning en la consola, pero no a nivel de usuario. **3.A** Si ocurre un error durante la petición (como un fallo de red o un error del servidor), se muestra una notificación de error. **4.A** Si el usuario no está autenticado (logueado en la página web), el servidor rechaza la petición y la extensión muestra una notificación indicando que es necesario iniciar sesión. **4.B** Si ninguna entrada del historial de lectura, coincide con alguno de los libros almacenados en el sistema, el servidor responde con un error y la extensión muestra una notificación indicando que no se pudo sincronizar (con que haya un libro sincronizable, esta excepción no salta).
Comentarios	- Si no hay capítulos, el botón de sincronización manual aparece deshabilitado.

Tabla 34: Extensión de navegador - CU-02 (Sincronizar historial de lectura)

CU-03	**Mostrar compatibilidad**
Precondición	- La extensión está instalada y habilitada en el navegador
Dependencias	A-01 (Usuario) A-03 (Background) RF-02 (Visibilidad del sitio soportado)
Descripción	Cambia el icono de la extensión según la página actual pertenezca a un sitio web soportado o no.
Secuencia normal	

	PASO	**ACCIÓN**
	1	El usuario navega por internet: cambia de pestaña, abre una nueva o la url de la existente cambia.
	2	El background verifica si la url de la pestaña actual pertenece a alguno de los sitios soportados (incluyendo la web principal).
	3.1	Si se trata de un sitio soportado, el icono de la extensión mostrado en dicha pestaña es el original (en verde).
	3.2	Si no es un sitio soportado, el icono de la extensión mostrado en dicha pestaña es uno diferente (en rojo), indicando que no está soportado.
Postcondición	El icono de la extensión, refleja si el sitio actual está o no soportado.	
Comentarios	- Por limitaciones, el icono no se podrá actualizar en páginas internas del navegador.	

Tabla 35: Extensión de navegador - CU-03 (Mostrar compatibilidad)

CU-04	**Mostrar información de la extensión**
Precondición	- La extensión está instalada y habilitada en el navegador
Dependencias	A-01 (Usuario) RF-04 (Sincronización manual) RF-08 (Visualización de capítulos por sincronizar)
Descripción	Permite al usuario mostrar una ventana emergente con información de la extensión; el listado de los libros (historial de lectura) por sincronizar y un botón para realizar la sincronización manual.
Secuencia normal	

	PASO	**ACCIÓN**
	1	El usuario hace clic en el icono de la extensión
	2	Se abre un popup mostrando la lista de capítulos por sincronizar y el botón de sincronización manual.
Postcondición	Se abre el popup de la extensión.	
Comentarios	- El botón de sincronización manual solo está disponible en caso de que el usuario se encuentre en alguna página web perteneciente al dominio de la página web del sistema.	

Tabla 36: Extensión de navegador - CU-04 (Mostrar información de la extensión)

3. Visión general del sistema

En este apartado se presenta una descripción global del sistema desarrollado, abarcando tanto la arquitectura general como la interacción entre sus componentes principales. También incluye un diagrama ilustrativo que resume gráficamente el diseño modular adoptado.

3.1. Arquitectura y relación entre componentes

El sistema desarrollado se organiza en tres componentes principales, cada uno encargado de una funcionalidad concreta:

- **Sistema de Web Scraping:** se encarga de extraer información actualizada sobre las obras de sitios webs externos. Esta información se almacena en la base de datos del servidor.

- **Plataforma Web:** centraliza la información, permite al usuario gestionar su biblioteca y consultar su progreso.

- **Extensión de navegador:** detecta el capítulo que el usuario está leyendo y guarda el progreso localmente. Para evitar una sobrecarga de peticiones al servidor, la sincronización no se realiza tras cada capítulo leído, sino que se pospone (almacenándolo en el almacenamiento local del navegador) y ejecuta automáticamente cuando el usuario vuelve a la página principal.

Esta arquitectura evita acoplamientos directos entre la extensión y el scraper, manteniendo la plataforma web como núcleo de coordinación y almacenamiento.

3.2. Diagrama de componentes

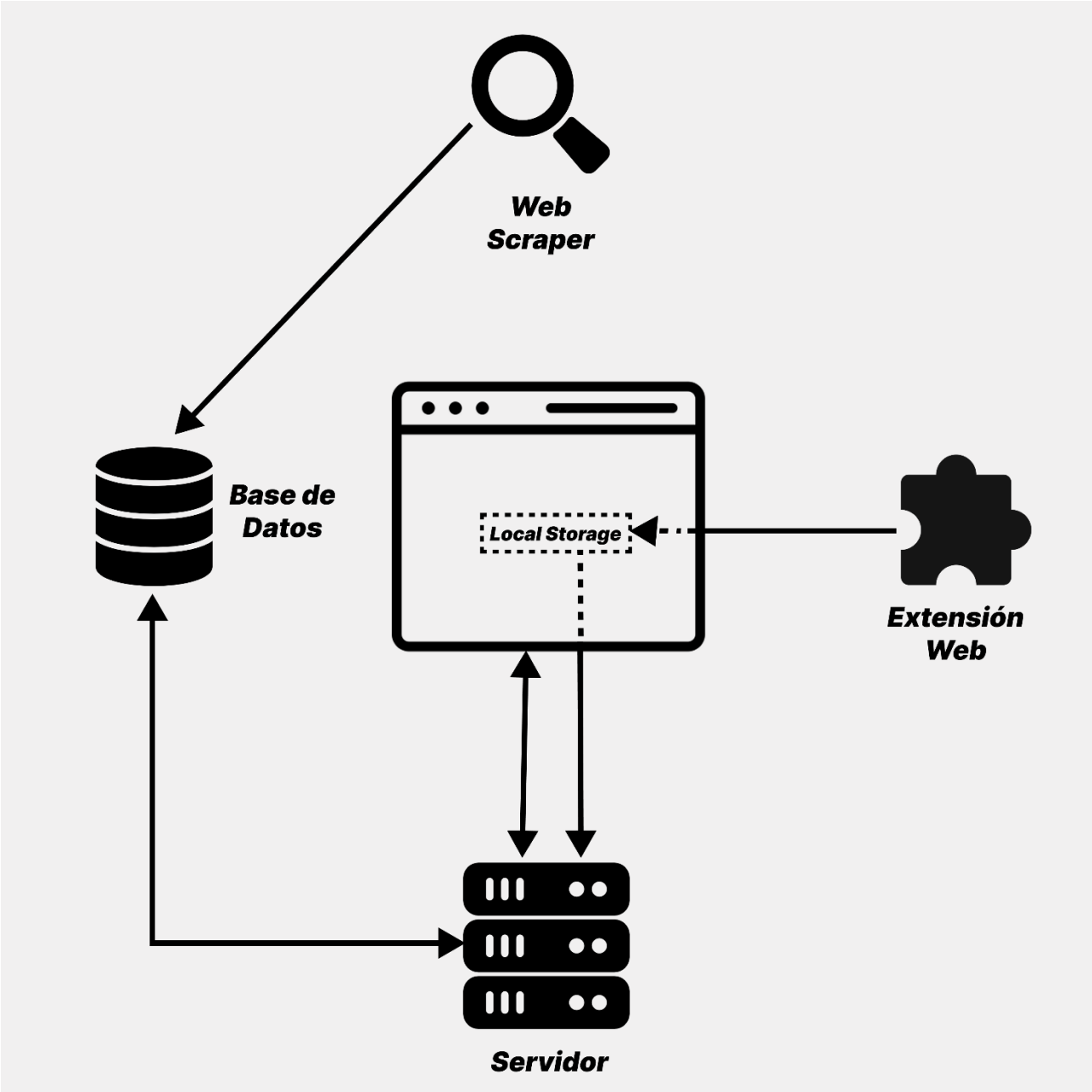

Ilustración 7: Diagrama de componentes

4. Tecnologías utilizadas

El desarrollo de este proyecto ha requerido del uso de múltiples tecnologías, tanto para el diseño y modelado previo como para la implementación de los distintos componentes. En este apartado se describen las principales herramientas, lenguajes, frameworks y plataformas empleadas, organizadas según su función dentro del proyecto.

4.1. Herramientas de diseño y modelado

Antes de empezar la implementación del sistema se emplearon herramientas para diseñar tanto la interfaz gráfica como los modelos conceptuales mediante diagramas UML (Larman, 2012).

4.1.1. Figma

Figma (Figma, Inc., s.f.) es una herramienta de diseño de interfaces basada en la nube que permite crear prototipos interactivos, wireframes y diseños colaborativos en tiempo real. Se utilizó para el diseño inicial (prototipos) de la interfaz web del sistema.

4.1.2. Astah UML

Astah UML (Change Vision, Inc., s.f.) es una herramienta de modelado visual de software, especializada en la creación de diagramas UML. Se utilizó para generar los diagramas de clases, de casos de uso y de secuencia del sistema

4.2. Recursos gráficos

Para mejorar la interfaz del proyecto y facilitar el diseño de iconos se utilizaron bibliotecas especializadas.

4.2.1. Flaticon

Flaticon es una plataforma con una amplia colección de iconos vectoriales que permiten enriquecer la experiencia visual de las aplicaciones. Se usó para obtener iconos adaptados a la identidad gráfica del sistema.

4.2.2. Font Awesome

Font Awesome es una librería de iconos muy popular que proporciona iconos escalables y personalizables mediante CSS. Se integró para facilitar la inclusión de iconos en la página web y en la extensión de navegador.

4.3. Tecnologías Web

Para desarrollar tanto la web como la extensión, se utilizaron tecnologías web básicas para estructurar el contenido, aplicar estilos y añadir interactividad a las páginas.

4.3.1. HTML

HTML (HyperText Markup Language) es el lenguaje de marcado utilizado para estructurar el contenido de la web. En el proyecto se empleó para construir tanto la plataforma web como la extensión de navegador.

4.3.2. CSS

CSS (Cascading Style Sheets) es un lenguaje de diseño web que permite dar estilo visual a los elementos HTML. Fue utilizado para definir el diseño, colores y disposición de la plataforma web y la extensión del navegador.

4.3.3. JS

JavaScript es un lenguaje de programación orientado a eventos, esencial en la creación de interfaces interactivas en la web. Se utilizó tanto en la plataforma web como en la extensión de navegador.

4.4. Herramientas de desarrollo

Durante el desarrollo del proyecto se emplearon herramientas auxiliares que facilitaron tanto la edición del código como la gestión del control de versiones.

4.4.1. Pycharm

Pycharm (JetBrains, s.f.) es un entorno de desarrollo integrado (IDE) especializado en Python (Python Software Foundation, 2025). Se utilizó como editor principal para centralizar el desarrollo del proyecto, permitiendo trabajar de forma centralizada con todos los componentes del sistema en un único entorno.

4.4.2. Github

GitHub es una plataforma de alojamiento de código en la nube basada en Git. Se utilizó principalmente como sistema de control de versiones y como respaldo del código. Además, se vinculó con Render (Render, s.f.) para facilitar el despliegue de la aplicación.

4.5. Frameworks y librerías

El desarrollo del sistema se apoyó en frameworks que facilitan la creación de aplicaciones web robustas y escalables, así como de librerías especializadas en web scraping.

4.5.1. Django

Django es un framework web de alto nivel para Python que promueve el desarrollo rápido y limpio. Se usó como base del backend, manejando rutas, modelos, lógica del servidor y administración del sitio.

4.5.2. Scrapy

Scrapy (Scrapy developers, 2025) es un framework de scraping de alto nivel para Python, diseñado para automatizar el rastreo de sitios web y extraer información estructurada de forma eficiente. Su arquitectura basada en spiders permite definir procesos de crawling y dirigir los datos a pipelines para su procesamiento y almacenamiento.

4.5.3. Playwright

Playwright es una herramienta de automatización de navegadores desarrollada por Microsoft que permite simular la interacción de un usuario real con sitios web. En este proyecto, no se utilizó directamente, sino a través de una librería (scrapy-plugins, 2025), la cual permite integrarlo con Scrapy. Esta integración se empleó para scrapear sitios web cuyo contenido se renderiza dinámicamente mediante JavaScript, o que requieran acciones simuladas por el usuario (como desplazamiento o eventos de click) para desencadenar la carga de información adicional.

4.6. Bases de datos

La información del sistema, tanto la generada por los usuarios como la obtenida mediante scraping, se almacena de forma estructurada en una base de datos relacional.

4.6.1. PostgreSQL

PostgreSQL es un sistema de gestión de datos relacional y de código abierto, conocido por su robustez y escalabilidad. Se utilizó para guardar tanto el contenido generado del scraping como datos relacionados con la plataforma web como los usuarios.

4.7. Plataformas de despliegue

Una vez finalizado el desarrollo, se utilizaron servicios en la nube para alojar la aplicación y hacerla accesible a través de internet.

4.7.1. Render

Render es una plataforma en la nube que permite desplegar aplicaciones web y servicios backend. En este proyecto se utilizó por su bajo coste y facilidad de uso, así como por su buena integración con aplicaciones desarrolladas con Django

5. Componente 1: Sistema de Web Scraping (Scrapy)

El sistema de Web Scraping es el primer componente del proyecto y se encarga de automatizar la extracción de información de obras desde sitios webs externos. Este componente ha sido construido con una arquitectura robusta que facilita la extensión, mantenimiento y escalabilidad para dar soporte a nuevas fuentes de datos.

5.1. Introducción técnica

El sistema de Scraping desarrollado en este proyecto se basa en el framework Scrapy, una herramienta de alto nivel para Python, que permite automatizar el rastreo de sitios web y extraer información de forma eficiente. En lugar de escribir scripts específicos para cada sitio web, se ha

implementado una arquitectura extensible basada en clases que modelan el comportamiento de cada página.

Scrapy permite separar responsabilidades mediante componentes especializados, con:

- **Spiders**, encargados de recorrer las páginas y extraer los datos

- **Pipelines**, que procesan y almacenan la información obtenida

- **Middlewares**, que permiten modificar el comportamiento de las peticiones y respuestas

Para sitios que cargan contenido de forma dinámica mediante JavaScript, o que requieren interacción del usuario (como hacer scroll o pulsar botones), se integró Playwright mediante la librería scrapy-playwright. Esto permite simular un navegador real únicamente cuando es necesario, manteniendo la eficiencia de Scrapy en los casos restantes.

La solución se ha diseñado para ser fácilmente escalable y mantenible: agregar soporte para nuevos sitios o adaptarse a cambios en el HTML requiere un esfuerzo mínimo, gracias a la arquitectura modular implementada.

Frente a alternativas como BeautifulSoup o Selenium, se optó por Scrapy debido a su rendimiento, capacidad de concurrencia y arquitectura modular. BeautifulSoup, aunque simple de utilizar, carece de funcionalidades como la gestión de paginación, paralelismo o interacción directa con navegadores. Selenium, aunque más potente en interacción, requiere de más recursos y es menos eficiente en tareas masivas. Scrapy representa un equilibro entre control, velocidad y escalabilidad para las necesidades de este proyecto.

5.2. Estructura del Scraper

La arquitectura de sistema de scraping ha sido diseñada para aprovechar la modularidad y escalabilidad de Scrapy.

- **Spiders**. El proyecto cuenta con dos spiders principales, cada uno orientado a un caso de uso distinto.

    - WebSiteScraper: spider encargado de recorrer todo el catálogo de un sitio web y extraer todos los libros que éste contiene (caso de uso CU-1).

    - DetailsPageSpider: spider especializado en procesar directamente la pagina de detalles de un libro individual, permitiendo actualizar datos concretos sin la necesidad de recorrer el catálogo completo (caso de uso CU-2).

  Estos no solo se usan para extraer los datos sino que también para validarlos, pudiendo ser redirigidos al sistema de reintentos en caso de faltar campos considerados como obligatorios.

Ambos spiders utilizan instancias concretas de clases que heredan de AbsPage, adaptadas a la estructura HTML de cada sitio web soportado.

- **Items**. En el archivo ítems.py se definen los wrappers que contienen la información extraída durante el scraping (BookItem, ChapterItem).

- **Pipelines**. Los ítems generados por los spiders se envían al pipeline, componente que centraliza la lógica de almacenamiento. El pipeline utilizado se define en el archivo de configuración settings.py,

- **Middlewares**. El sistema emplea los middlewares por defecto de Scrapy.

- **Excepciones**. Se dispone de un archivo llamado exceptions.py donde se define una excepción específica para los casos de datos incompletos. Aunque su uso actual es limitado, esta estructura permite una posible ampliación futura del sistema de gestión de errores.

5.3. Gestión de errores

Dado que el scraping web está expuesto a múltiples factores de fallo (cambios en el HTML, errores de red, timeouts, etc), el sistema incorpora una lógica de gestión de errores y reintentos diseñada para aumentar su robustez.

- **Reintentos automáticos**. Aunque Scrapy incluye por defecto un mecanismo de reintentos ante fallos (como errores 500, 502, timeouts, etc), se ha extendido dicha funcionalidad mediante un manejado de errores personalizado 'handle_error', capaz de actuar ante una mayor cantidad de errores y que da hasta 3 oportunidades antes de marcar la petición de scrapeo como un fallo y ser registrado en el sistema de logs de errores (failed_urls.txt).

- **Casos que activan el sistema de reintentos**:

    - Timeouts debidos al renderizado de Playwright.

    - Corrupción en los datos scrapeados, producidos debido a la falta de campos considerados como obligatorios (para esto se usa la excepción personalizada definida en exceptions.py, llamada ScrapedFieldsError)

    - Cualquier otro error interno de Scrapy.

5.4. Inserción en la base de datos (pipeline)

Una vez que los datos scrapeados han sido validados dentro del spider, el item 'BookData' se envía al pipeline 'SaveBookDataPipeline', encargado de su persistencia en la base de datos.

Este pipeline asocia cada libro a su Scanlator correspondiente (modelo ‘Scanlator’) y guarda o actualiza tanto la información del libro (modelo ‘Book’) como la de sus capítulos (modelo ‘Chapter’). Para ello se utiliza el método update_or_create, que permite actualizar una entrada existente o crearla si no existe, evitando duplicidades en la base de datos.

En caso de que el Scanlator no exista en la base de datos, el sistema detendrá la ejecución del spider de forma controlada. Si ocurre un error inesperado durante la inserción, también se interrumpirá el proceso de scraping, registrando el fallo en los logs para su posterior análisis.

5.5. Proceso para dar soporte a un nuevo sitio

El sistema ha sido diseñado con una arquitectura modular y extensible que facilita la incorporación de nuevos sitios web de forma sencilla y mantenible. La lógica de extracción de datos está organizada en torno al patrón de diseño Strategy, donde cada sitio web es tratado como una estrategia independiente de scraping.

Cada sitio se implementa como una clase concreta que hereda de AbsPage, una clase abstracta que define una interfaz común que deben seguir todas las implementaciones. Esto permite intercambiar dinámicamente la estrategia de extracción en función del sitio objetivo, sin modificar el flujo general del sistema.

El proceso para dar soporte a un nuevo sitio web consta de los siguientes pasos:

1. **Verificar que el sitio sea scrapeable**

   Antes de comenzar con la implementación, es necesario comprobar que el sitio web no esté protegido por servicios como Cloudflare, reCAPCHA o mecanismos que bloqueen el acceso automatizado. En caso de detectar tales barreas, el sitio será descartado.

   Para probar un sitio, basta con igualar el atributo ‘TESTING_PAGE_URL’ con la url de la página web objetivo y ejecutar el método ‘run_acceptance_test()’ .

2. **Identificar la estructura objetivo**

   Para agregar un nuevo sitio web, es fundamental entender su estructura de navegación y extracción. Todos los sitios compatibles comparten un mismo patrón funcional dividido en dos pasos (a continuación veremos un ejemplo de extracción para el scanlator ‘Comikey’):

   - **Página del catálogo**: es la página donde se listan los libros disponibles. Esta página permite obtener las urls de detalles de cada libro así como la siguiente página en caso de haber paginación.

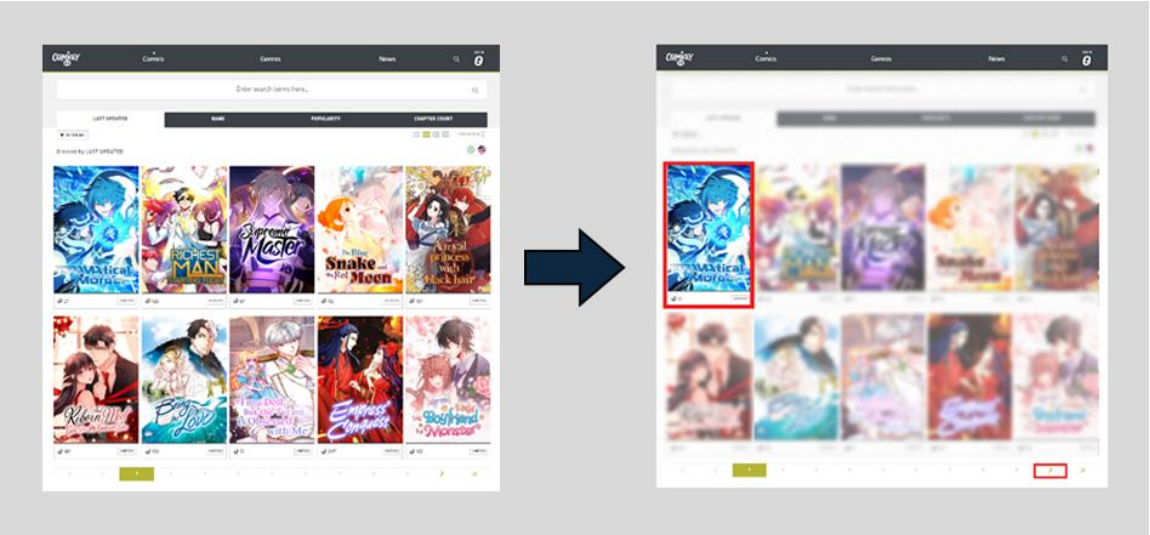

Ilustración 8: Ejemplo de elementos a scrapear en la página del catálogo

- **Página de detalles**: es la página que contiene toda la información extraíble de un libro concreto: titulo, autor, sinopsis, imagen, listado de capítulos, etc.

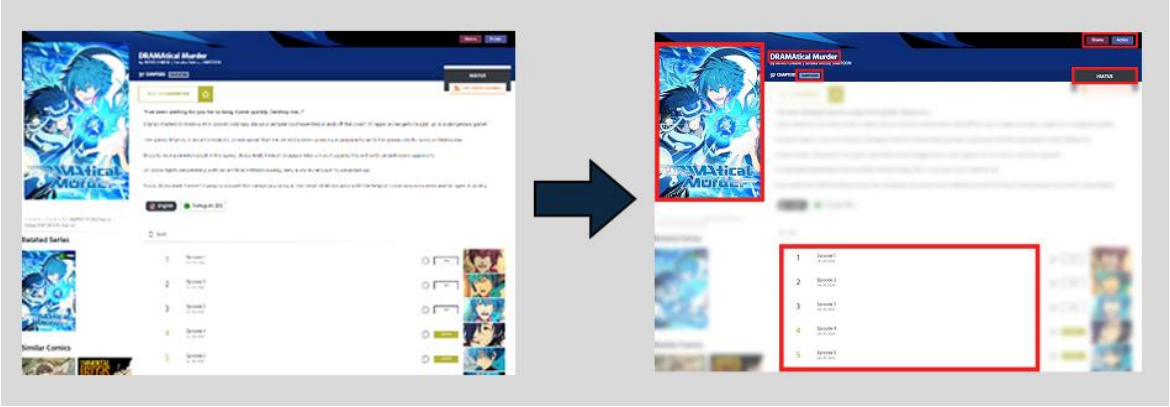

Ilustración 9: Ejemplo de elementos a scrapear en la página de detalles

Esta separación en dos fases garantiza una estructura común para todos los sitios soportados, permitiendo aplicar una misma lógica base al flujo general del sistema. Es requisito indispensable que la url de la página de detalles se pueda obtener o construir (mediante ingeniería inversa) desde la página del catálogo.

3. Rellenar la plantilla de scraping en JavaScript (opcional)

Se utiliza una plantilla escrita en JavaScript como paso intermedio para modelar la lógica de extracción. Esta plantilla se rellenará de forma progresiva a medida que se vayan

identificando los selectores necesarios (css y xpath) para extraer los datos requeridos desde el navegador, ayudándonos de las herramientas para desarrollados que estos incorporan.

Este enfoque no solo nos ayudará en la depuración de errores (al no tener la necesidad de correr el scraper completo) sino que también nos permitirá generar el código en Python.

4. **Generar la clase de scraping**

Una vez completada y verificada la plantilla de JavaScript, el siguiente paso consiste en implementar la clase concreta que hereda de AbsPage, encargada de definir la lógica de extracción para el nuevo sitio.

Se empezará definiendo el constructor, el cual cuenta con:

- **webID**: el identificador único para el sitio web (se suele usar el host de la página web)

- **base_url**: la url de la página del catálogo (en caso de haber paginación, estará en la primera página).

- **Flags de Playwright**: flags para modelar el comportamiento en caso de que la página requiera de usar Playwright para poder scrapear todos los datos necesarios.

Página del catálogo	Página de detalles
mainPage_needPlaywright	detailsPage_needPlaywright
mainPage_playwright_needScroll	detailsPage_playwright_needScroll
mainPage_scrollCustomElement	detailsPage_scrollCustomElement
	detailsPage_playwright_needClick

Tabla 37: Flags de comportamiento para la integración con Playwright

A continuación, se procederá con la implementación de los métodos ‘getChaptersPageUrls(…)’ y ‘getChaptersData(…)’, la cual puede hacerse manual, utilizando la plantilla como guía. Sin embargo, se recomienda generar el código automáticamente mediante una IA (como chatGPT), proporcionando como entrada la plantilla del nuevo sitio junto con un ejemplo ya implementado.

Aunque este enfoque acelera el desarrollo, siempre será necesario revisar el código generado para asegurar que cumpla con los estándares del proyecto.

5. **Agregar a la lista de sitios soportados**

   Tras generar la clase concreta que hereda del AbsPage, es necesario instanciarla dentro de la lista ‘pagesList’, que contiene todos los sitios actualmente soportados por el sistema.

6. Componente 2: Plataforma Web (Django)

Este componente constituye el núcleo del sistema, proporcionando la interfaz web desde la que los usuarios pueden gestionar su biblioteca personal y consultar su progreso de lectura. Ha sido desarrollando con el framework Django, aprovechando sus capacidades integradas para el manejo de usuarios, la administración de datos y la separación entre lógica de negocio y presentación. La plataforma se conecta con una base de datos común, lo que permite acceder a la información recopilada por el scraper y mantener sincronizado el estado de lectura gracias a la extensión del navegador.

6.1. Introducción técnica

La plataforma web desarrollada para este proyecto está basada en Django, un framework de alto nivel para Python, diseñado para facilitar el desarrollo de aplicaciones web seguras, escalables y mantenibles. A diferencia de otros frameworks más flexibles o minimalistas, Django adopta una filosofía de “convención sobre configuración”, proporcionando una arquitectura clara y componentes bien definidos que guían el desarrollo.

A nivel estructural, Django impone una organización interna del código en distintos archivos con responsabilidades bien diferenciadas:

- **Models.py**: define la estructura de los datos mediante clases que representan las tablas de la base de datos.

- **Views.py:** contiene la lógica que responde a las peticiones del usuario.

- **Urls.py:** configura las rutas de acceso de cada vista.

- **Forms.py:** permite definir formularios y validaciones.

- **Admin.py:** registra y configura los modelos para el panel de administración (operaciones CRUD).

Esta división favorece la claridad del proyecto.

6.1.1. Arquitectura del framework (MTV)

Django empela su propia variante del clásico patrón MVC (Model – View – Controller), conocido como MTV (Model – Template – View):

- **Modelo (Model):** define la estructura de datos de la aplicación mediante el sistema ORM. Cada modelo es una clase que representa una tabla de la base de datos, con sus campos y relaciones.

- **Vista (View):** gestiona la lógica de negocio. Recibe las peticiones HTTP, consulta los datos necesarios y devuelve una respuesta, normalmente en forma de plantilla renderizada o como respuesta JSON.

- **Plantilla (Template):** define la interfaz que verá el usuario. Django utiliza su propio motor de plantillas con soporte para herencia, filtros y estructuras de control.

Este modelo no solo separa la lógica del negocio y la presentación, sino que facilita pruebas y el mantenimiento.

6.1.2. Sistema de persistencia (ORM)

Una de las fortalezas clave de Django es su ORM (Object-Relational Mapping). Gracias a este sistema, es posible trabajar con la base de datos sin escribir SQL directamente. Algunas de sus características son:

- Traducir clases Python en tablas SQL automáticamente.

- Permitir definir relaciones entre modelos (ForeignKey, ManyToManyField, etc).

- Soportar migraciones automáticas (con makemigrations y migrate).

- Permitir hacer consultas complejas de forma expresiva (Model.objects.filter(…)) y segura (impidiendo inyecciones SQL).

Este sistema no solo acelera el desarrollo, sino que también protege frente a inyecciones SQL, ya que todas las consultas se parametrizan internamente.

6.1.3. Funcionalidades integradas y ventajas de Django

Django incluye una serie de herramientas que cubren funcionalidades comunes para el desarrollo web, sin necesidad de integraciones externas. Algunas de las más relevantes para este proyecto son:

- **Panel de administración automático**: al registrar los modelos en ‘admin.py’, se genera automáticamente una interfaz web para gestionar los datos con búsqueda, filtros, formularios y validaciones.

- **Sistema de autenticación**: incluye la gestión de usuario, registro, login, logout, recuperación de contraseñas, permisos y grupos.

- **Sistema de formularios**: los formularios pueden generarse automáticamente a partir de los modelos o de forma personalizada (forms.py), con validaciones tanto del lado del cliente (parciales) como del servidor.

- **Protecciones de seguridad integradas**:

    - Protección contra inyecciones SQL gracias al ORM.

    - Tokens CSRF automáticos en todos los formularios, para evitar ataques de falsificación de peticiones.

    - Escapado automático de variables en las plantillas para evitar ataques XSS.

- **Sistema de middlewares**: capas intermedias que permiten interceptar, modificar o analizar las peticiones/respuestas HTTP, como por ejemplo, para autenticar usuarios, redirigir rutas, controlar cabeceras, etc.

- **Desarrollo de APIs REST**: mediante el uso de librerías como Django REST Framework (DRF), se pueden construir y exponer endpoints estructurados, seguros y fácilmente testeables.

En conjunto, Django ofrece una solución coherente y robusta que cubre desde el modelo de datos hasta la presentación, pasando por la lógica de negocio y las operaciones de backend más comunes.

6.2. Interfaces de usuario (prototipos)

Los siguientes diseños representan una versión preliminar de la interfaz web. Sirvieron como base durante el desarrollo, aunque algunos elementos no se incorporaron en la versión final.

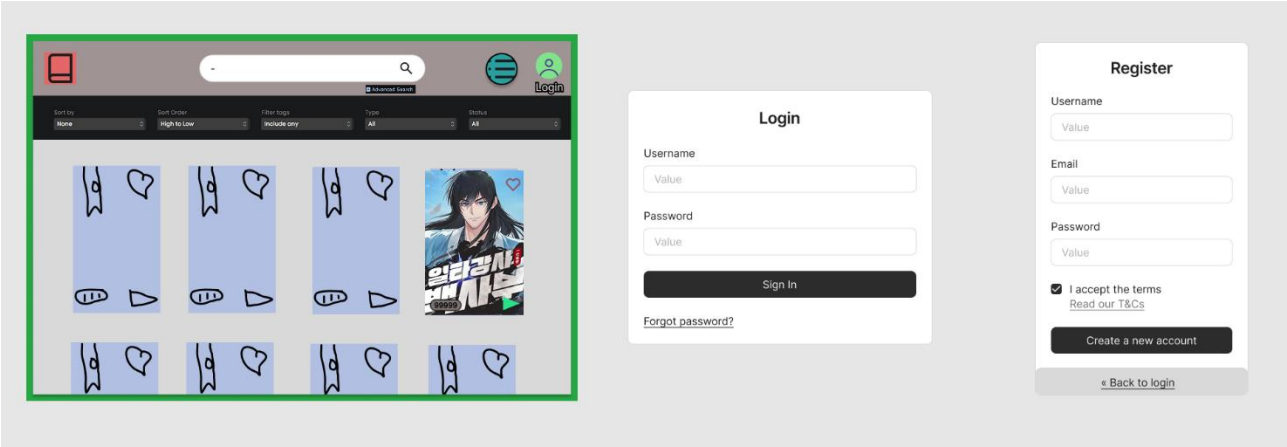

Ilustración 10: Prototipos de la página principal, del login y del registro de usuario

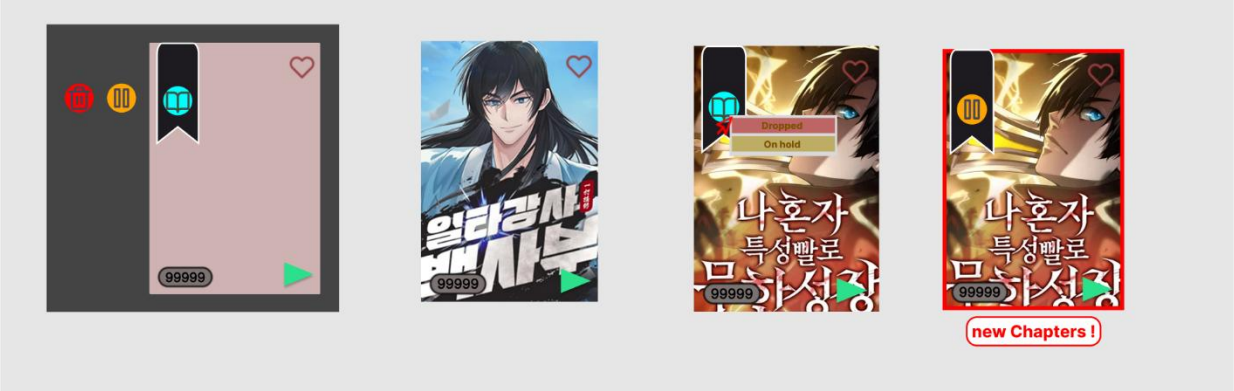

Ilustración 11: Prototipos de la carátula de un libro

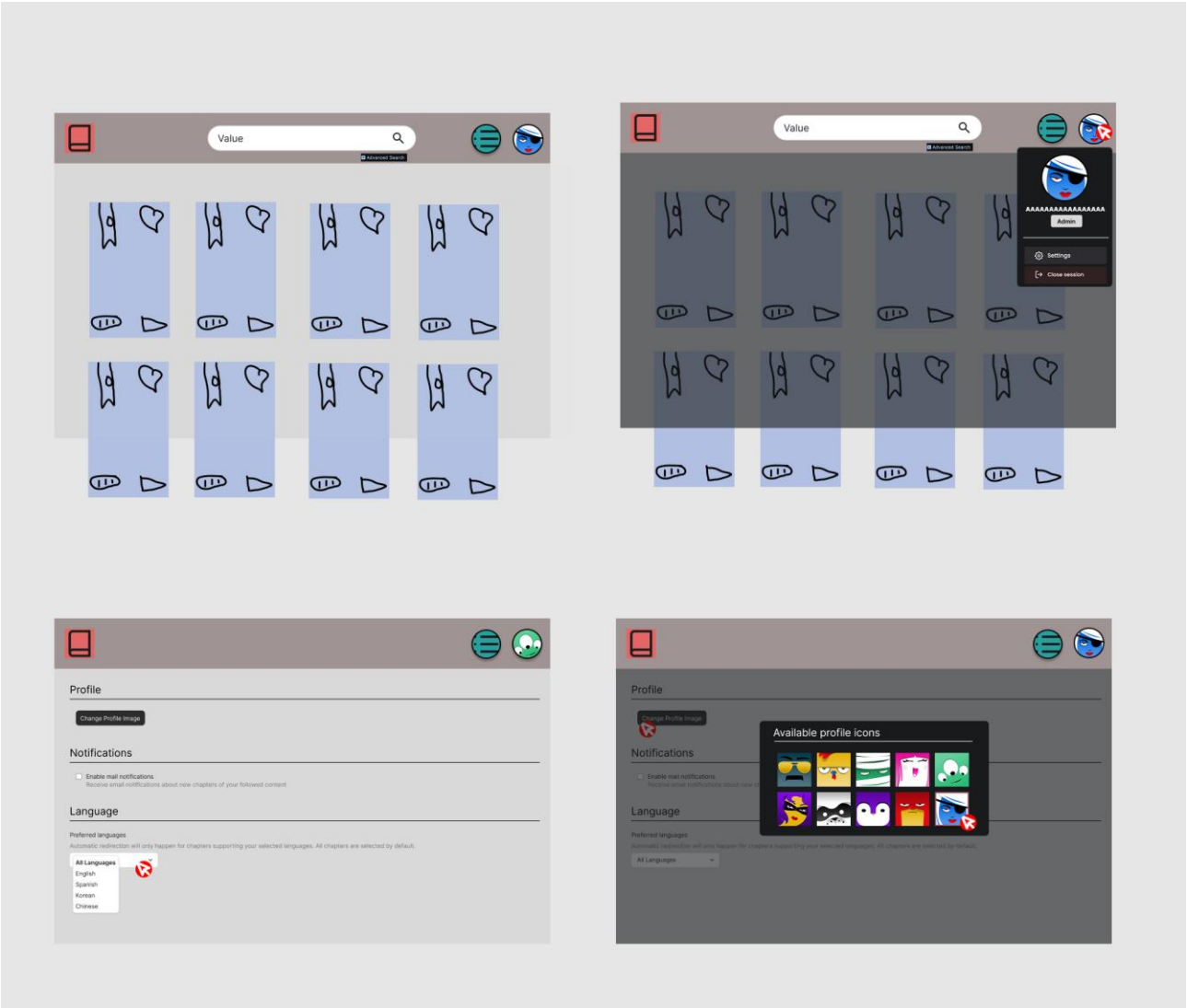

Ilustración 12: Prototipos para las vistas de la configuración de la cuenta

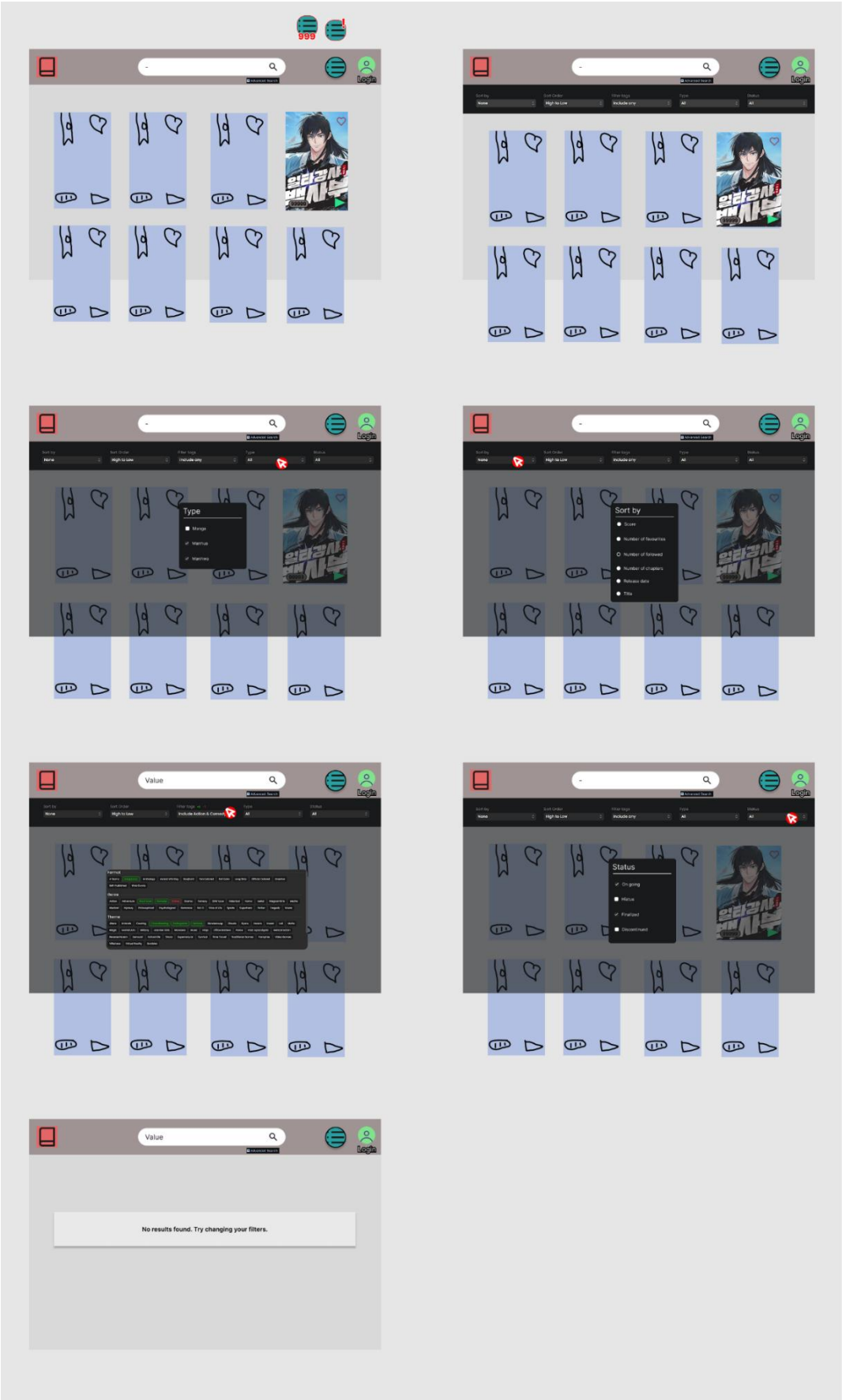

Ilustración 13: Prototipos para las vistas de los filtros de búsqueda

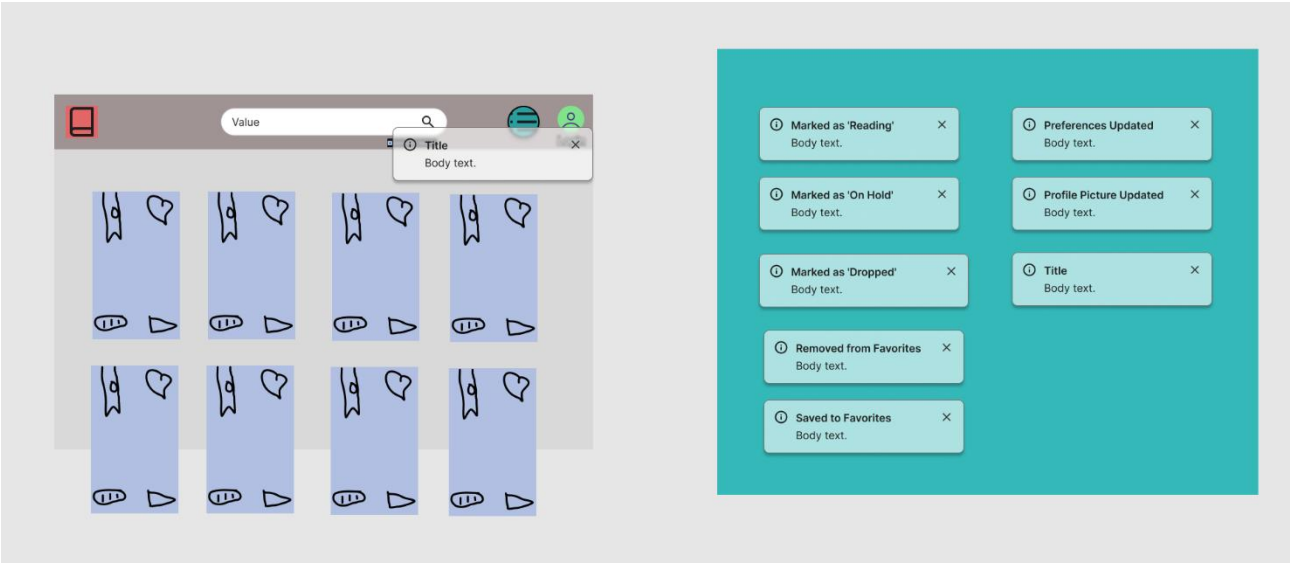

Ilustración 14: Prototipos de las notificaciones y mensajes posibles

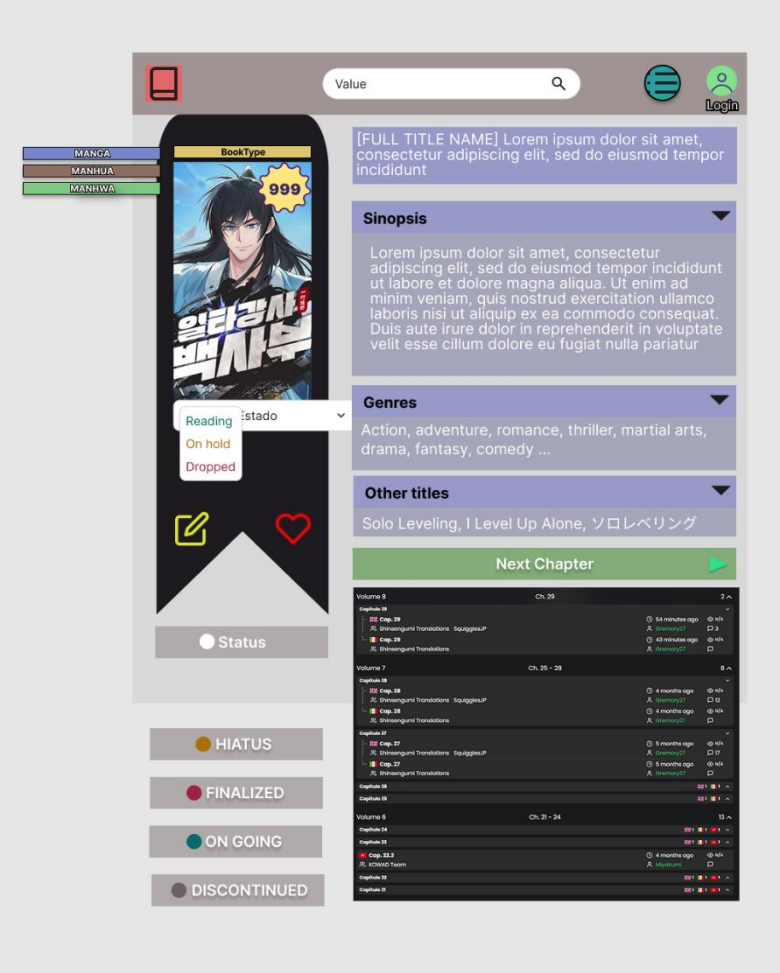

Ilustración 15: Prototipo de la página de detalles de un libro

6.3. Modelado de base de datos

A continuación se detallan las entidades principales, las relaciones entre ellas y las decisiones de diseño adoptadas durante la implementación.

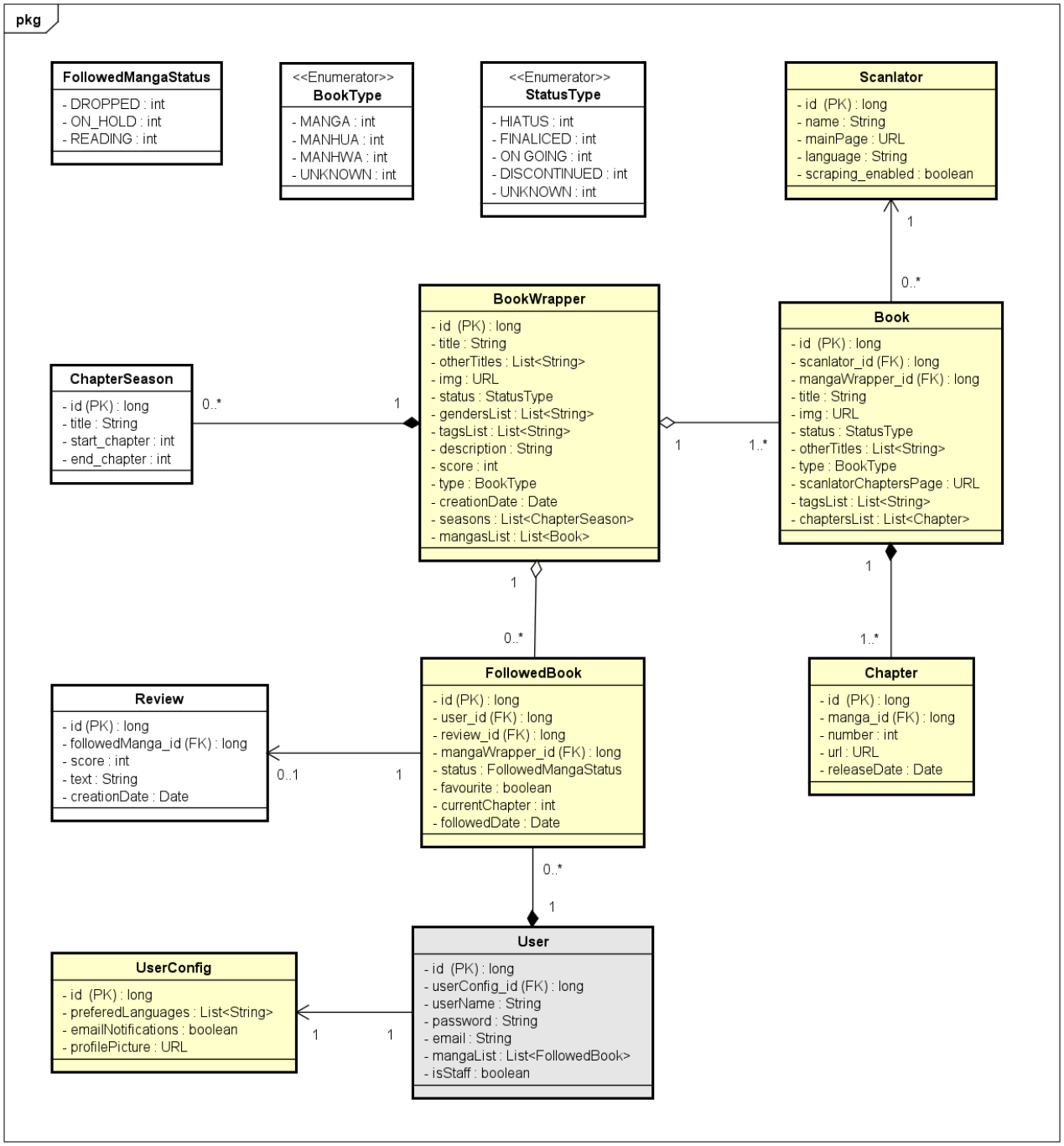

Ilustración 16: Diagrama UML del modelado de la base de datos

6.3.1. Modelo de usuario

Aunque en el modelo UML inicial se planteó una clase “User” con los atributos mostrados en el diagrama anterior, en la implementación final se ha optado por reutilizar el modelo de usuario proporcionado por Django. Esta decisión permite aprovechar todo el sistema de autenticación ya implementado (registro, login, logout), evitando así la gestión manual de contraseñas y sesiones.

La configuración adicional del usuario se ha separado en la tabla “UserConfig”, relacionada mediante una relación uno a uno con el modelo “User”. De esta manera, se mantiene la lógica limpia y modular, almacenando las configuraciones por separado.

6.3.2. Relación entre libros y fuentes

El núcleo del sistema gira en torno a los scanlators, es decir, los sitios web que publican obras en línea. Dado que una misma obra puede estar licenciada por múltiples scanlators y cada uno puede tener versiones distintas, se introducen las siguientes entidades:

- **Book**: representa una versión concreta de una obra publicada por un único scanlator. Contiene información como su título, estado, tipo, lista de etiquetas, etc.

- **BookWrapper**: actúa como una agrupación de libros (Book) que, aunque provienen de distintas fuentes, representan la misma obra. Esta entidad contiene información agregada como otherTitles, tagsList, genresList, o la sinopsis. Esta duplicación de datos, aunque no ideal desde el punto de vista de la normalización, se ha decidido conscientemente para reducir la carga computacional al acceder a la vista de detalles, evitando operaciones de agregación y filtrado en tiempo real.

- **Scanlator**: representa una fuente de publicación. Incluye información básica como su nombre, la url de la página principal, el idioma y un flag “scraping_enabled” que permite desactivar temporalmente el scraper para dicha fuente en caso de errores críticos.

La relación entre Book y BookWrapper es una agregación, para que en caso de borrar un wrapper, los libros individuales se mantengan. Lo mismo ocurre entre BookWrapper y FollowedBook, como medida de protección para conservar los registros de lectura aunque se eliminen obras del sistema.

6.3.3. Capítulos y progreso

Los capítulos (Chapter) están ligados directamente al Book y no al BookWrapper, ya que cada scanlator tiene URLs distintas para los capítulos y en muchos casos también cuentan con capítulos faltantes.

Por otro lado, para representar el seguimiento de un usuario sobre una obra, se introduce la clase FollowedBook. Esta relación contiene información como el estado de lectura (Reading, On Hold, Dropped), si es favorito, el número del último capítulo leído, etc. Se relaciona directamente con el modelo User (mediante composición, con borrado en cascada) y con BookWrapper (mediante agregación).

6.3.4. Otros aspectos del modelo

Existen varias decisiones de diseño adiciones que conviene destacar:

- Se introdujeron enumerados (BookType, BookStatus, FollowedStatus) para estandarizar ciertos campos. Sin embargo, debido a la enorme variabilidad del contenido extraído mediante scraping (por ejemplo, distintos nombres como “Manhua” y “WebComic”, ”Finalized” y ”Finished”, etc.), se optó finalmente por permitir cualquier valor en estos campos, para no limitar la recogida de información.

- En general, las fechas no tienen un papel relevante en el funcionamiento del sistema y, dado que su formato varía enormemente según el scanlator, se ha optado por almacenarlas como texto plano (String) en lugar de como objetos (Date), con el fin de evitar problemas de conversión o parseo.

- El atributo otherTitles permite almacenar múltiples títulos alternativos que una misma obra puede tener, fruto de traducciones a diferentes idiomas o variaciones hechas por el propio scanlator. Esto es útil para facilitar tanto la agrupación de libros bajo un mismo BookWrapper, como para ayudar al usuario a encontrar una obra.

- También se definió la tabla ChapterSeason, que permitía agrupar capítulos en temporadas, pero que no ha sido implementada en esta versión del proyecto, al estar alejada del objetivo principal del TFG.

- De igual forma, la tabla Review, que permitía que los usuarios valorasen las obras que siguen, tampoco ha sido utilizada en la versión final, quedando reservada para una posible ampliación futura del sistema.

6.4. Interfaz de fusión de libros

Fusionar Libros

Libros Disponibles

Buscar libro | Buscar

COMIKEY	I'm a Doll, but the Tyrant Is Obsessed With Me	Agregar a fusión
COMIKEY	The Blue Snake and The Red Moon	Agregar a fusión
COMIKEY	My Darling Signed In	Agregar a fusión
COMIKEY	Reborn And Meet, Give You My Exclusive Love	Agregar a fusión
COMIKEY	Richest Man Starting Today	Agregar a fusión
TAPAS	They Say I Was Born a King's Daughter	Agregar a fusión
TAPAS	The Villainess is a Marionette	Agregar a fusión
TAPAS	Precious Daughter of the Greatest Martial Arts Villain	Agregar a fusión
TAPAS	Taming Master	Agregar a fusión

BookWrappers Disponibles

Buscar BookWrapper | Buscar

NUM. BOOKS	TÍTULO	ACCIÓN
1	Damned Demonic Swords	Agregar a fusión
1	Lout of Count's Family	Agregar a fusión
1	All STR Sorcerer	Agregar a fusión
1	Ascend	Agregar a fusión
1	A Returner's Magic Should Be Special	Agregar a fusión
1	I Can See Your Death	Agregar a fusión
1	Magician's Murim Adventures	Agregar a fusión
1	Dungeon Majesty	Agregar a fusión

Libros Seleccionados para Fusionar

SCANLATOR	TÍTULO	ACCIÓN
COMIKEY	DRAMAtical Murder	Eliminar

Generar Opciones

Seleccionar Valores para la Fusión

Título:
[COMIKEY] DRAMAtical Murder

Imagen:
[COMIKEY] DRAMAtical Murder

Estado:
HIATUS

Tipo:
Webtoon

Descripción de Wrapper:
-- Seleccionar descripción --

Descripción:
Ingrese una descripción

Guardar Fusión

Ilustración 17: Interfaz del sistema de fusión de libros

6.5. Mapas de navegación

A continuación, se presenta el mapa de navegación de la aplicación web, que representa las distintas rutas y funcionalidades accesibles según si el usuario se ha logueado o no en el sistema.

Para usuarios no logueados:

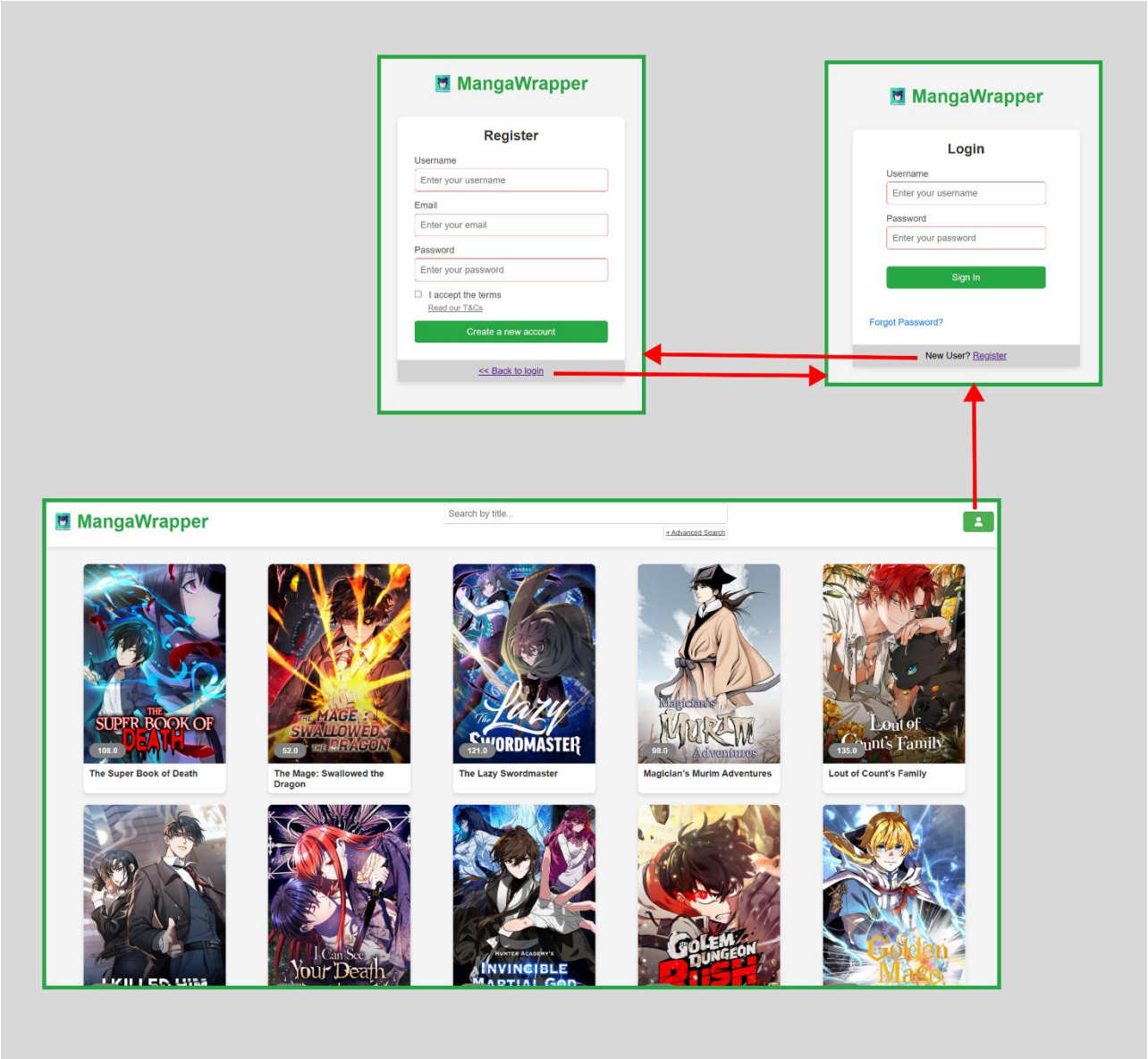

Ilustración 18: Navegación para el logueo y registro de usuario

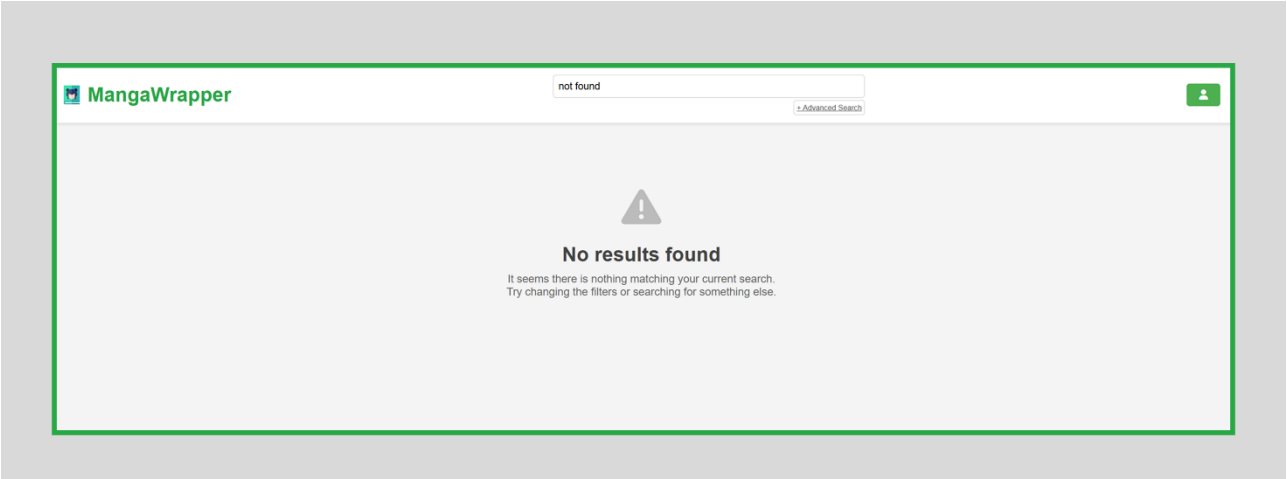

Ilustración 19: Mensaje de advertencia tras hacer una búsqueda sin resultados

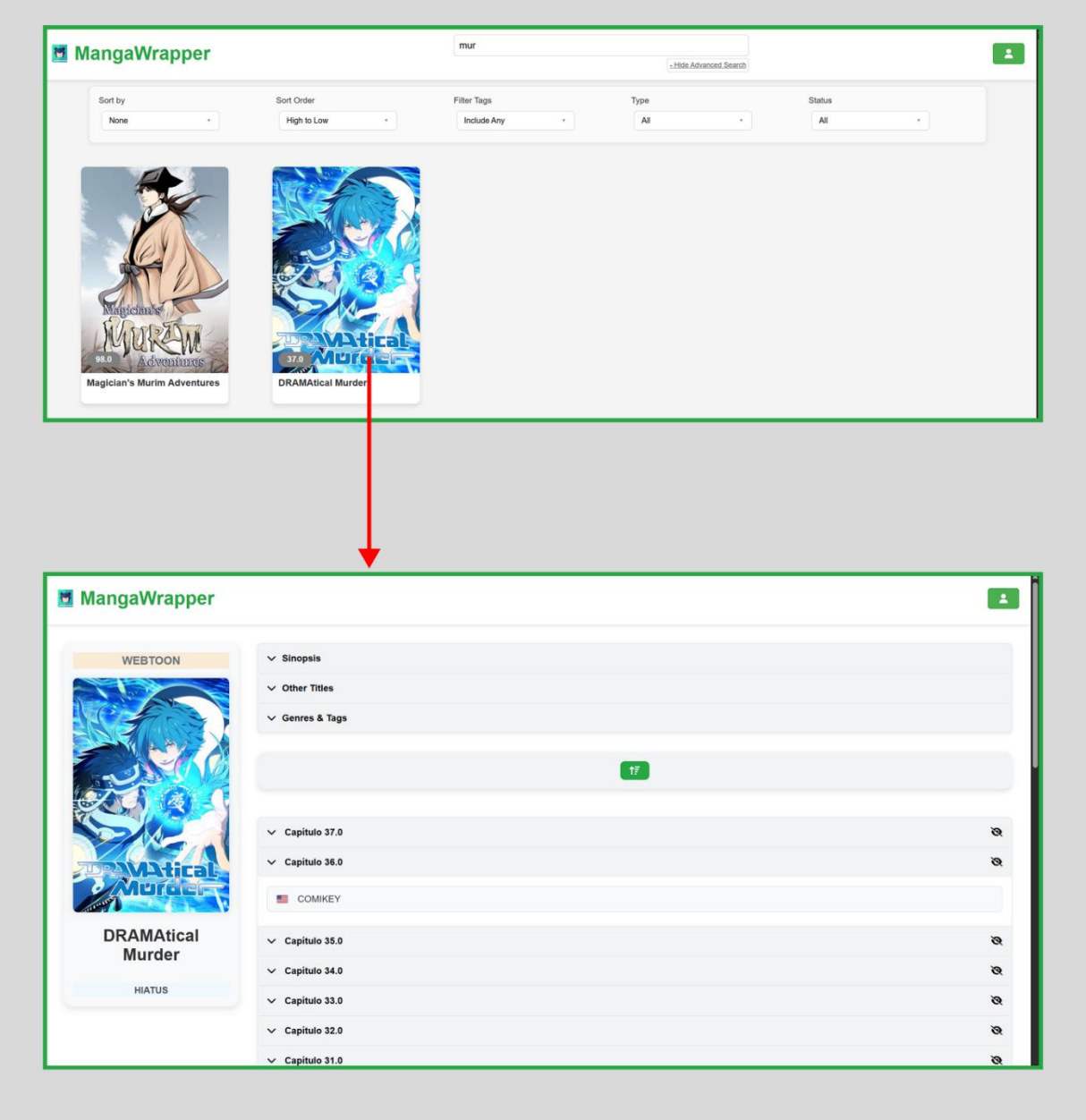

Ilustración 20: Navegación de un usuario no logueado a la vista de detalles

Para usuarios logueados:

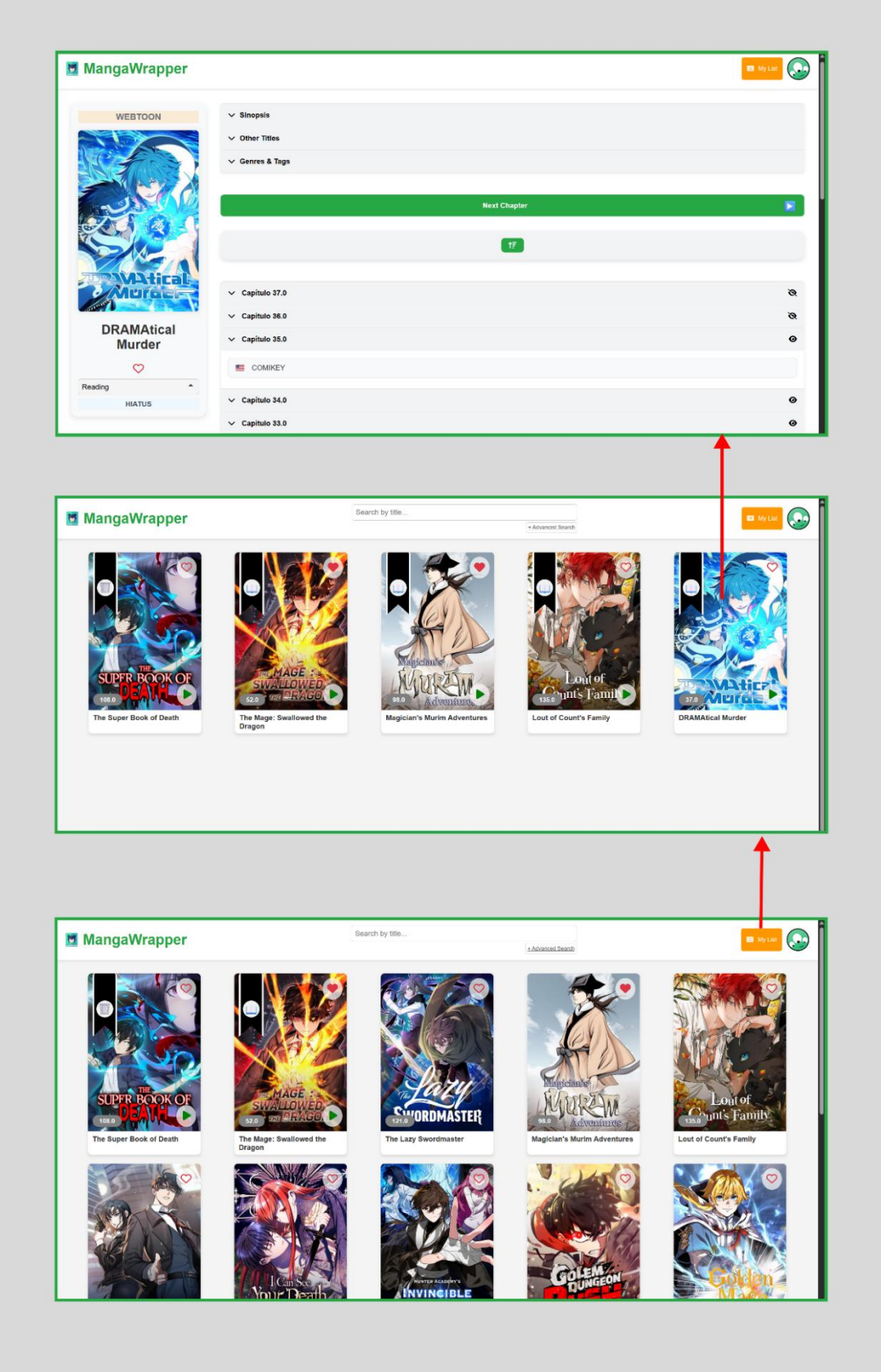

Ilustración 21: Navegación de un usuario logueado a MyList y a la página de detalles

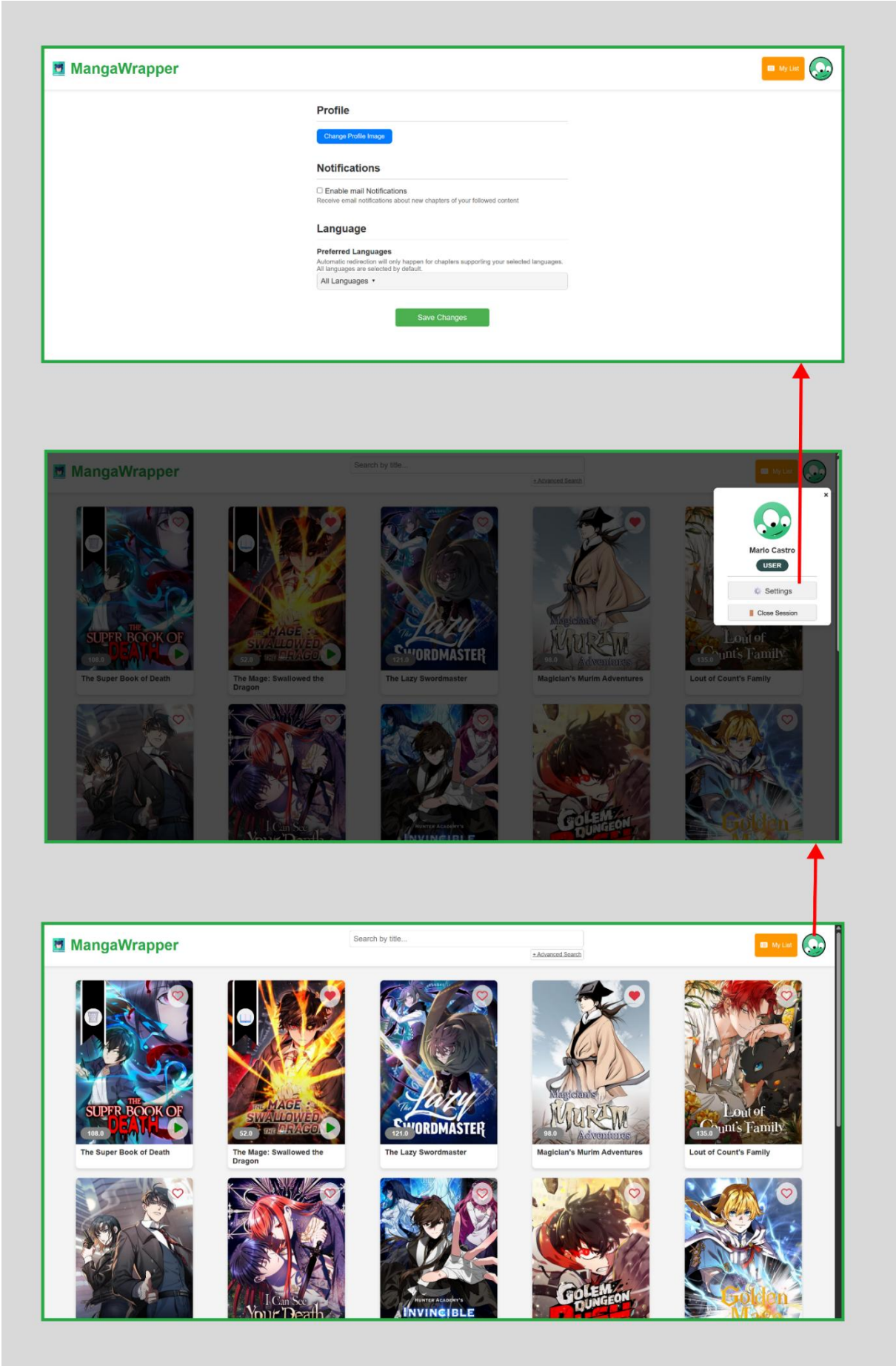

Ilustración 22: Navegación de un usuario logueado al panel de configuración de la cuenta

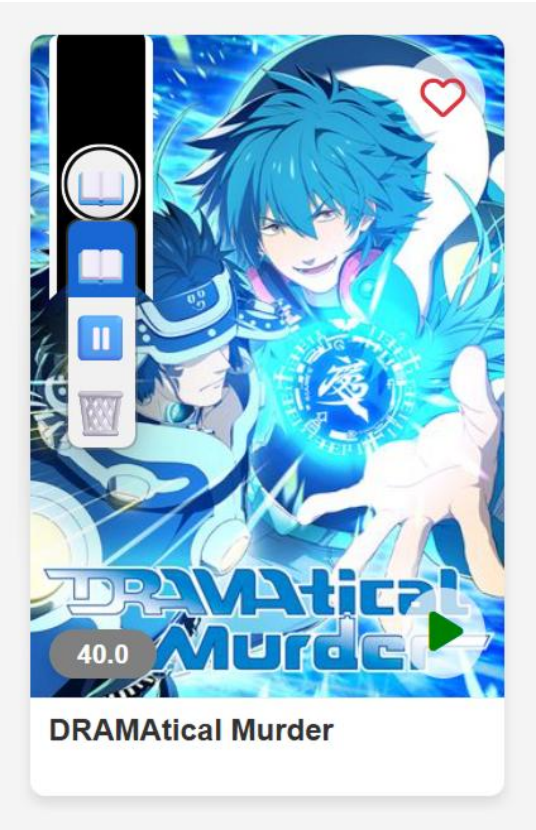

Ilustración 23: Interfaz para cambiar el FollowedStatus

6.6. Estructura de las plantillas

La aplicación no emplea una arquitectura basada en componentes dinámica, sino que sigue el patrón clásico de Django; usando plantillas estáticas con inclusión de fragmentos comunes mediante 'include'.

Para evitar duplicación de código, se han definido ciertos fragmentos HTML, los cuales modelan componentes completos o, simplemente, 'imports' para agilizar el desarrollo y dar claridad al código:

- **header.html**: componente que modela la cabecera. Permite activar o desactivar la barra de búsqueda mediante parámetros.

- **toastSystem.html**: se encarga de importar los recursos necesarios para mostrar notificaciones, mediante la librería 'toastr'.

- **i18.html**: habilita la internacionalización.

6.7. Limitaciones

A pesar de que el sistema cumple con su funcionalidad principal, hay ciertos aspectos mejorables o decisiones técnicas tomadas durante el desarrollo:

- **Comunicación con la extensión del navegador**: Para que el servidor acepte la petición de sincronización del historial de lectura de la extensión web, ha sido necesario desactivar la verificación del token CSRF en una vista concreta. Esta decisión se tomó tras múltiples intentos fallidos de configurar una alternativa compatible.

- **Ausencia de paginación en el catálogo principal:** La página principal no incluye paginación debido al tamaño reducido del catálogo.

6.8. API Básica

En este proyecto se ha desarrollado una API, aunque no es una API REST completa ni estándar. Se optó por una implementación más ligera y sencilla, adecuada a las necesidades del proyecto.

La API está compuesta por tres endpoints que permiten hacer acciones como: obtener la siguiente url de un libro en base a las preferencias lingüísticas del usuario, actualizar el progreso de lectura manualmente desde la página web y sincronizar el historial de lectura desde la extensión web.

7. Componente 3: Extensión de Navegador (Chromium)

La extensión de navegador es el componente encargado de detectar qué capítulo está leyendo el usuario en páginas web externas compatibles y registrar ese progreso. Opera de forma autónoma, almacenando datos localmente y sincronizándolos con el servidor cuando el usuario accede a la plataforma web. Su arquitectura modular permite dar soporte a nuevos sitios fácilmente.

7.1. Introducción técnica

La extensión de navegador desarrollada en este proyecto está diseñada para funcionar en navegadores basados en Chromium (como Google Chrome, Microsoft Edge, Brave, etc.). Su objetivo principal es registrar el progreso de lectura de los usuarios en los sitios web soportados y sincronizar dicho progreso con el servidor del sistema.

A nivel técnico, la extensión sigue la arquitectura recomendada por el manifiesto de Chrome (v3), separando la lógica en componentes especializados que cooperan mediante un sistema de mensajería interno:

- **Content Scripts**: son scripts que se ejecutan dentro del contexto de las páginas web y permiten acceder a su contenido (DOM) pero operan en un entorno aislado, lo que impide compartir directamente variables o funciones con el contexto de la web. Tienen acceso limitado a las APIs del navegador.

- **Background**: es un script que se ejecuta en segundo plano bajo demanda, gestionando eventos clave de la extensión. Tiene acceso completo a las APIs del navegador.

- **Popup**: es la interfaz gráfica que muestra al usuario al hacer clic en el icono de la extensión en la barra del navegador. Se suele usar como punto de acceso para que el usuario interactúe directamente con la extensión.

Estos componentes se comunican entre sí mediante el sistema de mensajería que proporciona Chrome (como chrome.runtime.sendMessage y chrome.runtime.onMessage).

Esta arquitectura impone ciertas restricciones, como la imposibilidad de usar módulos ES6 (imports/exports) en los content scripts, lo que dificulta la reutilización de funciones entre scripts. Además, el background no puede acceder directamente al DOM de las páginas web, lo que obliga a utilizar content scripts como intermediarios para interactuar con el contenido.

7.2. Interfaz de usuario

La interfaz de usuario se reduce a un pequeño popup (un tipo de ventana emergente) accesible al hacer clic sobre el icono de la extensión, el cual ofrece una visualización del historial de capítulos leídos (aún por sincronizar), junto a un botón que permite hacer la sincronización de forma manual.

Ilustración 24: Popup de la extensión al estar en la página principal

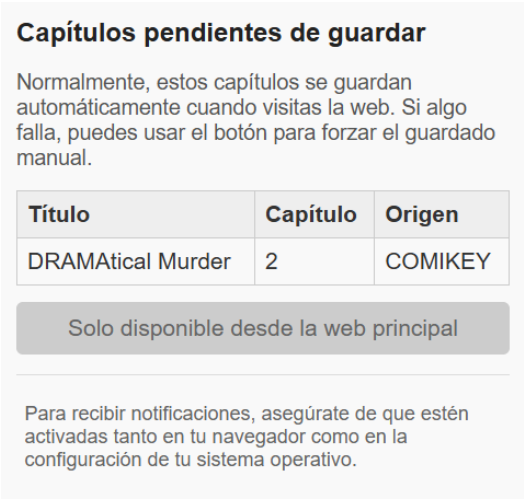

Ilustración 25: Popup de la extensión al estar en una página web externa

Ilustración 26: Icono de la extensión al estar en un sitio web soportado

Ilustración 27: Icono de la extensión al estar en un sitio web no soportado

7.3. Estructura de la extensión

La arquitectura de la extensión ha sido diseñada para aprovechar al máximo las capacidades de la API de extensiones de Chrome, manteniendo la modularidad y claridad en la responsabilidad de cada componente.

- **Content Scripts**: se utilizan para detectar las acciones del usuario sobre los sitios web soportados. En particular, permite capturar eventos como el cambio de capítulo, extraer la información del capítulo leído (título, numero de capítulo, scanlator) y desencadenar el registro del progreso de lectura.

- **Background**: es el encargado de gestionar el almacenamiento persistente del historial de lectura, mostrar notificaciones mediante la API del sistema operativo, controlar el cambio del icono de la extensión en función del dominio actual (mostrando si está o no soportado) y gestionar la comunicación con el servidor.

Consideraciones auxiliares:

- **Internacionalización (i18n)**: Para facilitar la traducción de la interfaz, se ha creado un script específico (i18n.js) que automatiza la asignación de textos localizados a los elementos del DOM con el atributo 'data-i18n'. Esto garantiza un comportamiento centralizado, evitando llamadas dispersas a la API 'chrome.i18.getMessage()', facilitando la escalabilidad. Actualmente se soportan las traducciones al español e inglés.

7.4. Proceso para dar soporte a un nuevo sitio

La extensión ha sido diseñada con una arquitectura modular y extensible que facilita la incorporación de nuevos sitios web de forma sencilla y mantenible. La lógica de sitios soportados, está basada en el patrón de diseño Strategy, donde cada sitio web es tratado como una estrategia independiente.

Cada sitio se implementa como una clase concreta que hereda de AbsCurrentPage, una clase abstracta que define una interfaz común que deben seguir todas las implementaciones. Esto permite seleccionar dinámicamente la estrategia de extracción de datos en función del sitio actual, sin modificar el flujo general del sistema.

El proceso para dar soporte a un nuevo sitio web consta de los siguientes pasos:

1. **Identificar los elementos a extraer**

   Para agregar un nuevo sitio web, es fundamental entender qué datos son necesarios extraer, para que la extensión tenga todas las funcionalidades. A continuación veremos un ejemplo de cómo dar soporte al scanlator 'Comikey':

   - **Página de lectura**: es la página donde ocurre la lectura de un capítulo. Todos los elementos necesarios para dar soporte a un sitio, serán extraídos desde esta página: el título del libro, el número del capítulo, los botones para pasar de capítulo y un elemento HTML que solo esté presente en la página de lectura (para evitar que se ejecuten scripts fuera de la misma)

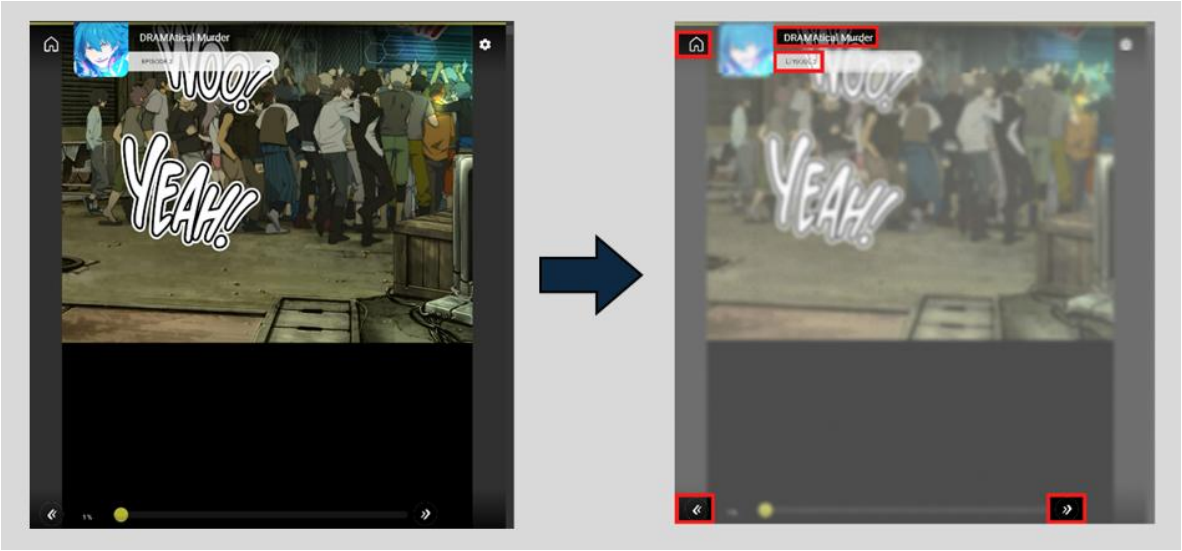

Ilustración 28: Ejemplo de elementos a extraer de la página de lectura

2. **Rellenar la clase plantilla**

   Para cada nuevo sitio se parte de una clase plantilla que define la estructura mínima necesaria. Esta clase hereda de AbsCurrentPage y actúa como guía para garantizar que todas las implementaciones sigan un mismo estándar.

   Gracias a esta plantilla, el proceso de adaptación a nuevos sitios se vuelve más rápido, predecible y fácil de mantener. Las modificaciones necesarias suelen limitarse a completar los fragmentos marcados en la plantilla, como lectores CSS.

3. **Agregar la clase a la lista de sitios soportados**

   Tras crear la clase concreta que hereda de AbsCurrentPage, es necesario instanciarla dentro de la lista 'SUPPORTED_SITES_LIST', que contiene todos los sitios actualmente soportados por el sistema.

   Esta lista se transforma automáticamente en un diccionario ('SUPPORTED_SITES_DICT') para agilizar el acceso en tiempo de ejecución

   (Debido al aislamiento entre los contextos del content script y el background, esta información se comparte mediante el sistema de paso de mensajes ('message passing') proporcionado por la API de Chrome.)

7.5. Limitaciones técnicas y soluciones adoptadas

Limitaciones derivadas de la arquitectura de Chrome:

- **Restricción de módulos ES6 entre Content Scripts**: los content scripts no pueden utilizar directamente módulos ES6 (import/export), lo que impide la reutilización directa de código entre archivos. Para mitigar esta limitación, se ha optado por inyectar al DOM aquel código compartido, haciendo posible su acceso por parte del resto de content scripts.

- **Acceso indirecto al DOM desde el background**: como el background no tiene acceso directo al DOM de las páginas web, se ha creado un componente específico (un content script) que actúa como puente (proxy) entre el background y el DOM. Este componente permite al background obtener información de la página cuando sea necesario, respetando la arquitectura de aislamiento de Chrome.

- **Aislamiento funcional del background**: el background no comparte el contexto con los content scripts, lo que obliga en ciertos casos a mantener copias duplicadas de funciones comunes. Para reducir los efectos negativos de esta duplicación, se ha optado por anotar en el código los scripts donde aparece dicha repetición, facilitando así su mantenimiento futuro.

Otros problemas encontrados:

- **Formato inconsistente del número de capítulo**: No solo cada sitio puede tener su propia forma de representar los capítulos (por ejemplo: Ch. 99, Capítulo 99, 99 – Nombre del capítulo), sino que incluso dentro de una misma página web pueden coexistir varios formatos entre libros distintos. Para evitar tener que definir funciones ad hoc para cada caso (lo cual sería difícil de mantener debido a la naturaleza cambiante de la web) se ha desarrollado una función genérica, basada en expresiones regulares que permite extraer de forma robusta el número del capítulo a partir de un texto.

8. Integración

La integración entre los distintos componentes del sistema ha sido sencilla gracias al uso de framework Django, que actúa como núcleo del proyecto y facilita la interacción entre sus partes:

- **Scraper:** Para que el scraper, que se ejecuta de forma independiente, pueda acceder a la base de datos del proyecto, es necesario configurar el entorno Django en dicho componente. Esto incluye:

    - Ajustar el PYTHONPATH para que incluya el directorio del proyecto de Django.

    - Establecer la variable de entorno DJANGO_SETTINGS_MODULE, para acceder al archivo de configuración de Django (settings.py).

    - Inicializar Django con django.setup().

Estas configuraciones permiten importar directamente los modelos definidos en Django y operar sobre la base de datos sin necesidad de configuraciones adicionales.

- **Extensión de navegador:** Para que la extensión pueda comunicarse con el servidor y sincronizar el historial de lectura, Django debe aceptar las peticiones de la extensión. Para ello, basta incluir la URL de la extensión en las listas CORS_ALLOWED_ORIGINS y CSRF_TRUSTED_ORIGINS de Django. Así, se permite el intercambio de datos autenticados a través de un endpoint protegido.

8.1. Problemas encontrados durante la integración

- **Compatibilidad entre Scrapy y Django:** Una de las principales dificultades fue la integración del pipeline de Scrapy con el ORM de Django en un contexto asincrónico; mientras que Scrapy es asíncrono, Django es síncrono. Para solucionar esto, se optó por envolver las peticiones síncronas de Django en asíncronas utilizando la función 'asgiref.sync.sync_to_async'.

- **Ejecutar Scrapy desde el panel de administración de Django:** Actualmente, el sistema de scraping se ejecuta desde el entorno de desarrollo, es por esto que se intentó implementar una solución que permitiera lanzarlo desde la página web, a través del panel de administración de Django. Sin embargo, por el momento esto no ha sido posible y en concreto se ven dos comportamientos diferentes dependiendo si la web a scrapear requiere o no el uso de Playwright:

    a. Si no requiere Playwright, el scraping se realiza, el problema ocurre justo después, cuantos datos se pasan al pipeline y lo que se observa es que nunca llegan a este.

    b. Si requieren Playwright, el sistema falla desde el principio, cerrándose sin hacer ningún scraping.

9. Despliegue

El despliegue se ha realizado en Render, utilizando tanto su sistema de alojamiento para aplicaciones web como su servicio de bases de datos.

El proceso se puede resumir en dos pasos:

1. Popular la base de datos en la nube desde el servidor local de Django.

2. Desplegar el servidor de Django en Render y conectarlo con esa base de datos.

Durante el despliegue surgieron varias situaciones relevantes:

- **Cambio de base de datos:** El paso de una base local a la remota fue sorprendentemente sencillo. Bastó con modificar una única línea de la configuración de Django y, como Scrapy utiliza ese mismo archivo, el cambio afectó a ambos.

- **Variables de entorno:** Render permite definir variables de entorno, lo que facilita ajustar ciertos parámetros, principalmente orientados a los ficheros de configuración, lo que permite hacer modificaciones directamente sin tener que tocar el código fuente.

También se detectó una limitación técnica importante:

- **Cuello de botella durante el guardado:** La base de datos de Render (alojada en Frankfurt y bajo el plan gratuito) presenta una velocidad de escritura muy limitada. Esto genera un cuello de botella: todos los hilos que deberían dedicarse a scrapear quedan bloqueados por aquellos que están escribiendo en la base de datos. En la práctica, esto provoca que el scraping se detenga sin razón aparente hasta que se guardan los libros recién scrapeados. Con esto, la velocidad aproximada de escritura de es de un capítulo por segundo.

A pesar de estas limitaciones, Render facilita el despliegue para proyectos de esta escala.

La página web desplegada es la siguiente: https://mangawrapper2.onrender.com/

10. Estimación de costes

El siguiente apartado detalla el coste estimado de desarrollo del proyecto desde una perspectiva empresarial.

10.1. Costes de personal

La mayor parte del coste del proyecto corresponde al tiempo dedicado por el personas técnico. En este caso, se ha estimado una dedicación media de 3 horas diarias durante 4 meses, considerando 30 días laborables al mes. El precio por hora utilizado es el de un ingeniero de software junior, tomando como referencia el mercado español:

Concepto	Detalle	Coste (€)
Desarrollo	360h x 15 €/h	5.400 €

Tabla 38: Costes de personal

10.2. Costes de software

Para el desarrollo han sido necesarias diversas herramientas de software:

Concepto	Detalle	Coste (€)
Astah UML	4 meses x 5,99 €/mes	23,96 €
Figma Profesional	4 meses x 20 €/mes	80 €
Pycharm	4 meses x 11,98 €/mes	47,92 €
Flaticon	12 meses x 7,50 €/mes	90 €
FontAwesome	1 año	75 €
		316.88 €

Tabla 39: Costes de software

10.3. Costes de hardware

Se ha estimado el coste de hardware mediante una amortización mensual del equipo necesario (ordenador, pantalla y periféricos) durante los cuatro meses del desarrollo:

Concepto	Detalle	Coste (€)
Amortización del hardware	4 meses x 50 €/mes	200 €

Tabla 40: Costes de hardware

10.4. Costes de infraestructura y despliegue

Se ha estimado el coste de despliegue para un periodo de un año:

Concepto	Detalle	Coste (€)
Render – Web Service (Standart)	12 meses x 25 €/mes	300 €
Render – PostgreSQL DB (pro-4gb)	12 meses x 55 €/mes	660 €
Dominio web	1 año de un dominio .com	12 €
		972 €

Tabla 41: Costes de infraestructura y despliegue

10.5. Costes operativos

Incluye una estimación de costes básicos como el internet y la electricidad:

Concepto	Detalle	Coste (€)
Internet	4 meses x 20 €/mes	80 €
Electricidad	360 horas x 0.063 €/kWh	22.68 €
		102,68 €

Tabla 42: Costes operativos

10.6. Coste total

A continuación se presenta el desglose completo de los costes directos del proyecto, junto con la estimación de los costes indirectos, calculados como un 15% del total de costes directos, conforme a los criterios habituales en el ámbito empresarial

Concepto	Detalle	Coste (€)
Costes de personal	Costes de desarrollo	5.400 €

Costes de software	Astah UML, Figma, Pycharm, Flaticon y FontAwesome	316.88 €
Costes de hardware	Amortizaciones	200 €
Costes de infraestructura y despliegue	Render y Dominio (1 año)	972 €
Costes operativos	Internet y electricidad	102,68 €
Costes Indirectos	Un 15% de los costes directos	1048.734
		8040.294 €

Tabla 43: Coste total

11. Conclusiones

Este TFG me ha permitido desarrollar una solución completa y funcional para centralizar el seguimiento de obras literarias leídas desde distintas plataformas. A través de un sistema compuesto por un scraper modular, una plataforma web y una extensión de navegador con sincronización automática, he logrado cubrir los objetivos planteados, aportado una solución real a un problema común entre lectores de cómics digitales.

Técnicamente, el sistema se apoya en tecnologías modernas como Django, Scrapy y Playwright, integradas en una arquitectura escalable y mantenible. Se ha puesto especial atención en aspectos como la separación de responsabilidades, la gestión de errores, la experiencia de usuario y la extensibilidad del sistema para soportar nuevas fuentes sin alterar la lógica principal.

Desde el punto de vista personal, este proyecto supone la culminación de una idea concebida al inicio de mi carrera. Ya durante un verano anterior, intenté abordar esta misma problemática, sin llegar a completarla ni alcanzar el nivel de profundidad técnica actual. Poder terminar este proyecto, surgido de una frustración personal como lector al tener disperso mi historial de lectura, ha sido especialmente satisfactorio.

Una de las premisas del diseño ha sido favorecer el escalado dinámico del sistema. La arquitectura implementada permite añadir nuevas fuentes de forma sencilla, reflejando una forma de programar que siempre he buscado: construir soluciones generalizables, ampliables y mantenibles.

El sistema aún puede mejorar. El principal cuello de botella es el tiempo necesario para añadir soporte a nuevas fuentes y la agrupación manual de obra en BookWrappers, debido a las diferencias estructurales entre sitios. Una posible mejora futura sería la incorporación de

herramientas de emparejamiento asistido por inteligencia artificial. También quedó pendiente la implementación de un sistema de reseñas y puntuaciones por parte de los usuarios.

En conjunto, este TFG me ha permitido crecer tanto a nivel técnico como personal. Al inicio tenía ciertas dudas sobre mi capacidad para afrontar un proyectos de esta envergadura, algo que podría asociarse al síndrome del impostor. Sin embargo, a medida que el desarrollo avanzaba y lograba integrar con éxito todos los componentes del sistema, esa percepción fue cambiando.

Para mí, programar me recuerda a montar en bici de montaña: no estás limitado al camino ya hecho; puedes abrir uno nuevo, aunque eso implique algún tropiezo. Haber completado este proyecto me ha demostrado que puedo crear mis propios caminos.

12. Bibliografía

Change Vision, Inc. (s.f.). *Astah Professional User Guide*. Obtenido de Astah: https://astah.net/support/astah-pro/user-guide/

Django Software Foundation. (2 de Abril de 2025). *Django 5.2 documentation*. Obtenido de Django Documentation: https://docs.djangoproject.com/en/5.2/

Figma, Inc. (s.f.). *Get started*. Obtenido de Figma Help Center: https://help.figma.com/hc/en-us/categories/360002051613-Get-started

Google. (s.f.). *Extensiones de Chrome*. Obtenido de Chrome Developers: https://developer.chrome.com/docs/extensions?hl=es-419

JetBrains. (s.f.). *PyCharm Help*. Obtenido de JetBrains Help: https://www.jetbrains.com/help/pycharm

Kniberg, H., Skarin, M., de Mary Poppendieck, P., & Anderson, D. (2010). *Kanban y Scrum: obteniendo lo mejor de ambos.* Estados Unidos de América: C4Media Inc.

Larman, C. (2012). *Applying UML and patterns: an introduction to object oriented analysis and design and iterative development.* Pearson Education India.

Mozilla. (s.f.). *CSS*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/CSS

Mozilla. (s.f.). *HTML*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/HTML

Mozilla. (s.f.). *JavaScript*. Obtenido de MDN Web Docs: https://developer.mozilla.org/es/docs/Web/JavaScript

Python Software Foundation. (15 de Abril de 2025). *Python 3.12 Documentation*. Obtenido de Python Documentation: https://docs.python.org/3.12/

Render. (s.f.). *Deploys*. Obtenido de Render Documentation: https://render.com/docs/deploys

Scrapy developers. (8 de Mayo de 2025). *Scrapy*. Obtenido de Scrapy Documentation: https://docs.scrapy.org/en/latest/

scrapy-plugins. (22 de Febrero de 2025). *GitHub*. Obtenido de scrapy-plugins/scrapy-playwright: https://github.com/scrapy-plugins/scrapy-playwright

13. Anexos

13.1. Uso de imágenes con fines educativos

En este proyecto se han utilizado imágenes de avatares de Netflix con fines exclusivamente ilustrativos y educativos. Estas imágenes son propiedad de Netflix y se emplean sin ánimo de lucro ni comercial. Se incluye esta aclaración para respetar los derechos de autor y garantizar la transparencia en el uso de recursos externos.

13.2. Glosario

Este glosario recoge definiciones de términos utilizados a lo largo de este documento.

- Book: versión concreta de una obra publicada en línea por un único sitio web. Contiene información como título, estado, tipo, etiquetas, sinopsis, capítulos, etc. Cada Book proviene de un único scanlator.

- BookWrapper: entidad que agrupa uno o varios Books que, aunque proceden de distintas fuentes, representan el mismo contenido (la misma obra). Su uso permite unificar múltiples versiones de la misma obra bajo un sola entrada en la interfaz, facilitando la navegación y evitando duplicados. Contiene datos agregados de los Books que contiene.

- Scanlator: en el contexto del proyecto, se utiliza este término para referirse a sitios web que publican versiones digitales de comics (manga, manhua, manhwa, etc.). Aunque el termino scanlator suele estar asociado a la distribución no oficial o pirata de obras, aquí se emplea únicamente como una etiqueta técnica para referirse a la fuente concreta del contenido.

- Scraping: proceso automatizado de extracción de información desde páginas web. En este proyecto se usa para obtener los datos de las obras (título, capítulos, géneros, etc) desde los scanlators, utilizando tecnologías como Scrapy y Playwright para gestionar tanto el contenido estático como dinámico.

- Popup: interfaz gráfica reducida que aparece al hacer clic en el icono de la extensión del navegador. Se utiliza como punto de entrada visual para que el usuario pueda consultar su historial de lectura o sincronizarlo manualmente.

- DOM (Document Object Model): representación estructurada del contenido HTML de una página web. Permite a los scripts de la extensión interactuar con los elementos de la página, como extraer el título del capítulo, identificar botones de navegación, etc.

- Token CSRF: Valor generado automáticamente por el servidor para proteger los formularios web frente a ataques de falsificación de peticiones.

- Endpoint: punto de acceso a una funcionalidad concreta de una API (Interfaz de Programación de Aplicaciones).

- Content script: script de una extensión que se ejecuta dentro del contexto de una página web.